MUHAMMAD NAWAZ SHARIF UNIVERSITY OF ENGINEERING AND TECHNOLOGY, MULTAN

LAB MANUAL

Virtual System and Services

Name

M.JUNAID

Reg. No

2023-IT-39

Degree

BS-IT

Session

2023-2027

Submitted to

MAM AYESHA ALAM

LAB Task 1

Fundamentals of Virtual Systems

Objective:

Define virtual systems and explain why virtualization is widely adopted.

Q1. What is a virtual system?

A virtual system (commonly called a Virtual Machine or VM) is a software-based imitation of a physical computer. It runs its own operating system and applications independently, even though it shares the physical resources (CPU, memory, storage) of a single host machine.

A piece of software called a hypervisor (e.g., VirtualBox, VMware) creates and manages these VMs. Each VM thinks it is running on real hardware, but in reality, it is isolated from other VMs and the host.

Real-world analogy:

Think of a large apartment building. The building itself is the physical server. Each apartment is a virtual machine - separate, private, and fully functional, but all share the same foundation, electricity, and water supply.

Why is this useful?

- **Cost efficiency** - One powerful server can host many VMs.
- **Isolation** - A crash or virus inside one VM does not affect others.
- **Flexibility** - Run different operating systems (Windows, Linux, mac OS) on the same physical machine.

Virtual Machine:

A Virtual Machine (VM) is a software-based computer that uses real hardware (CPU, RAM, disk) through a program called a hypervisor.

How a Virtual Machine Works - The Working Mechanism

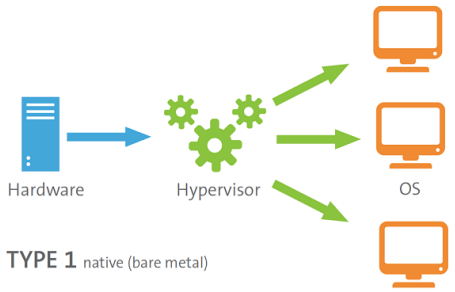
1. The Hypervisor: Types and Roles

The hypervisor (Virtual Machine Monitor, VMM) is the core software layer. It runs directly on hardware (Type 1) or on a host OS (Type 2).

Type 1 (Bare-metal):

Runs directly on physical hardware without a host OS.

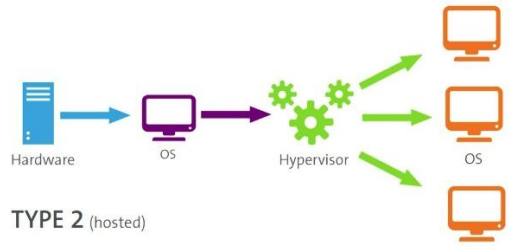
Examples: VMware ESXi, KVM, Microsoft Hyper-V. Used in data centres.



Type 2 (Hosted):

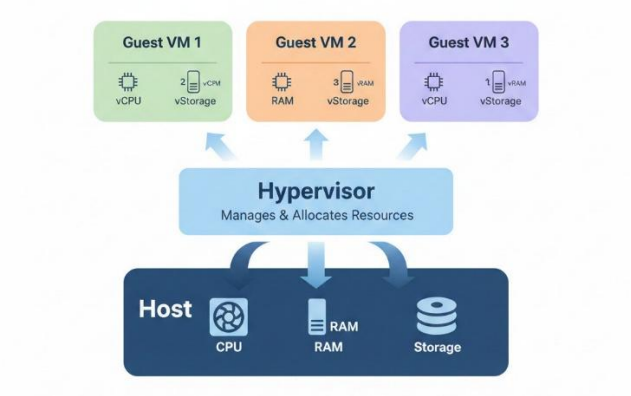
Runs as an application on top of Windows/Linux/macOS.

Examples: Oracle VirtualBox, VMware Workstation.



Hypervisor roles:

- Allocate physical CPU time, RAM, disk, and network to each VM.
- Isolate VMs so they cannot interfere with each other or the host.
- Translate virtual hardware requests into real hardware operations.



2. Step-by-Step Working of a VM

Step 1: Virtual Hardware Definition

The user (or management software) defines virtual hardware via the hypervisor's control panel:

- Number of virtual CPU cores (vCPUs)
- Amount of virtual RAM (e.g., 4 GB)
- Virtual hard disk size (e.g., 40 GB) as a file (.vdi, .vmdk, .qcow2)
- Virtual network adapter model (e.g., Intel PRO/1000)
- Other virtual devices (USB, sound, etc.)
- These are software settings stored in a VM configuration file.

Step 2: Guest OS Installation and Boot

The hypervisor presents the defined virtual hardware to the guest OS installer. The guest OS:

- Detects a standard CPU, RAM, disk, and network card.
- Loads drivers for those virtual devices.
- Has no knowledge that it is running on virtualised hardware.
- During boot, the hypervisor intercepts all hardware access attempts.

Step 3: CPU Execution

Problem: Guest OS expects exclusive control of the physical CPU.

Solution (modern - hardware-assisted):

Intel VT-x or AMD-V CPU extensions add two operation modes:

- Root mode - reserved for the hypervisor.
- Non-root mode - for guest VMs.
- Hypervisor sets up a Virtual Machine Control Structure (VMCS) defining which events cause VM exits.
- Most guest instructions execute directly on the physical CPU in non-root mode, at full speed.
- Only sensitive instructions (e.g., INVD, HLT, I/O port accesses) cause a VM exit to the hypervisor.
- Hypervisor handles the exit, then resumes the guest.

Result: Near-native performance (overhead <5%).

Scheduling: Hypervisor gives each VM a time slice (e.g., 10 ms) of physical CPU time. It saves the VM's CPU state (registers, program counter) on context switch, loads the next VM's state, and resumes.

Step 4: Memory Management - Address Translation

Problem: Each VM expects its own memory space starting at address 0. Physical RAM has one linear address space.

Solution - software (shadow page tables):

Hypervisor maintains a shadow page table mapping:

- Guest Physical Address (GPA) - what VM thinks is physical memory.
- Machine Physical Address (MPA) - real RAM location.
- Hypervisor intercepts page faults and updates tables.

Solution - hardware (Second Level Address Translation, SLAT):

- Intel EPT (Extended Page Tables) or AMD NPT (Nested Page Tables).
- Hypervisor sets up nested page tables once.
- CPU hardware automatically translates GPA → MPA on every memory access, without hypervisor intervention.

Result: Very low overhead, near-native memory performance.

Step 5: Device Emulation - I/O Handling

Problem: Directly exposing real hardware to a VM is dangerous (a buggy guest driver could crash the host) and reduces portability.

Solution: The hypervisor provides virtual devices (generic models). Process:

- Guest OS writes to virtual device's I/O port or DMA buffer.
- Access triggers a VM exit (or is intercepted).
- Hypervisor reads the command/data from virtual device memory.
- Hypervisor translates it into a command for the real physical device (e.g., network card, disk, USB).
- Real device performs operation; reply is returned to hypervisor.
- Hypervisor injects reply into virtual device's receive buffer and generates a virtual interrupt to the guest.

For disk: Guest read/write commands become file read/write operations on the host's virtual disk file (.vdi, .vmdk, etc.).

Benefits:

- **Portability** - VM can move to any host because it only sees generic virtual devices.
- **Protection** - A buggy driver crashes only the virtual device, not real hardware.

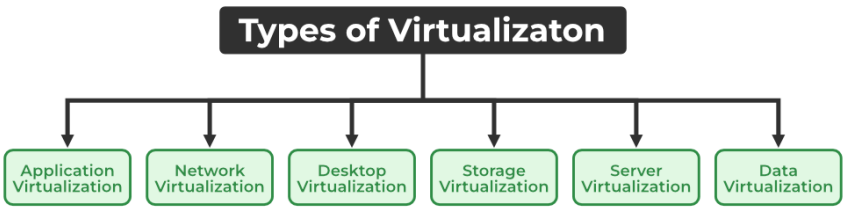
Virtualization:

Virtualization is a technology that uses software (called a hypervisor) to create one or more virtual versions of physical computing resources -- such as a computer (CPU, memory, storage), network, server, or operating system. These virtual versions, often called virtual machines (VMs) or virtual environments, run isolated from each other but share the underlying physical hardware.

Simple Analogy

Think of a large apartment building:

- The building is the physical computer.
- Each apartment is a virtual machine (VM).
- All apartments share the same building foundation, electricity, water supply, and parking - that's the physical hardware.
- But each apartment has its own private space, lock, and utilities - that's isolation.



Q2. Give two reasons why virtualization is necessary in modern computing.

1. Hardware utilization improvement

Physical servers typically run at 10-15% capacity. Virtualization allows multiple VMs to use the same hardware, pushing utilization to 70-80% or more. This reduces the number of physical servers, saving space, power, and cooling costs.

2. Safe environment for testing and development

Developers can test new software, security patches, or even malware inside a VM without risking the host system. If the VM breaks, it can be discarded or restored from a snapshot in minutes – no need to reinstall the host OS.

Q3. Provide a real-life example of virtualization.

Cloud-based virtual servers (e.g., AWS EC2, Digital Ocean Droplets).

When you rent a "virtual server" from a cloud provider, you are actually getting a VM running on one of their large physical servers. The provider uses virtualization to split a single powerful machine into hundreds of small virtual servers. You pay only for the resources you use (e.g., 1 CPU core, 2 GB RAM). To you, it feels like a dedicated physical server, but you are sharing hardware with many other customers. This makes cloud computing affordable and scalable.

LAB Task 2

Virtualization Types & Host Compatibility Check

Objective:

Learn different forms of virtualization and verify if your PC supports hardware virtualization.

Q1. List and briefly describe three types of virtualization.

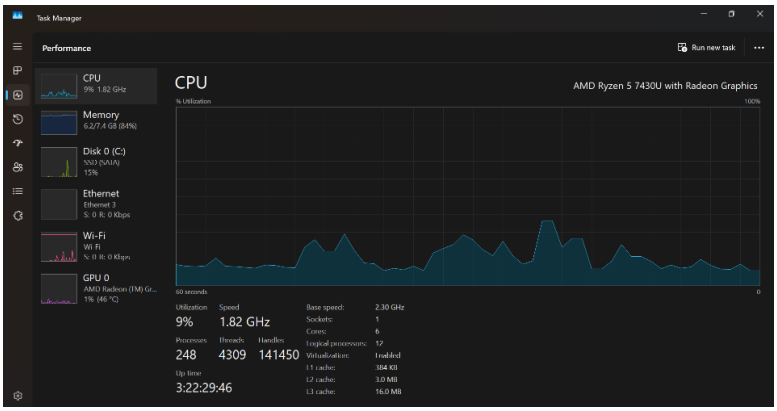
Types

- **Hardware (CPU) virtualization:** The hypervisor directly manages physical CPU and memory to run multiple guest OSes. Requires Intel VT-x or AMD-V.
- **Storage virtualization:** Combines multiple physical storage devices into a single logical storage pool (e.g., RAID, SAN).
- **Network virtualization:** Creates virtual networks (VLANs, virtual switches) that run on top of physical network hardware.

Q2. How to check if your Windows system supports virtualization?

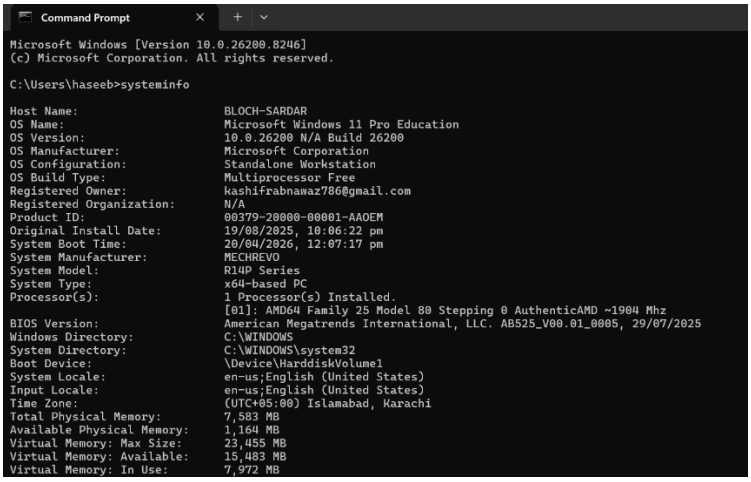
Method A – Using Task Manager

- Press "**Ctrl + Shift + Esc**" to open Task Manager.
- Go to the Performance tab and select CPU.
- Look at the bottom-right section: Virtualization - should say Enabled.

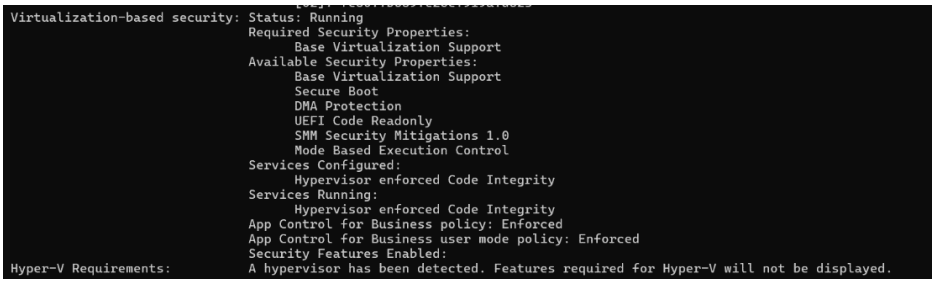


Method B – Using Command Prompt

- Open Command Prompt as administrator.
- Type "**systeminfo**" and press Enter.
- Scroll to the Hyper-V Requirements section.
- If you see Virtualization Enabled in Firmware: Yes, it is active.



If A hypervisor has been detected, it is already in use (e.g., by Windows Defender Application Guard).



If you see Disabled, you need to enter your computer's BIOS/UEFI (usually by pressing F2, Del, or F12 during startup) and enable Intel VT-x or AMD-V.

LAB Task 3

Installing VirtualBox and Obtaining OS Images

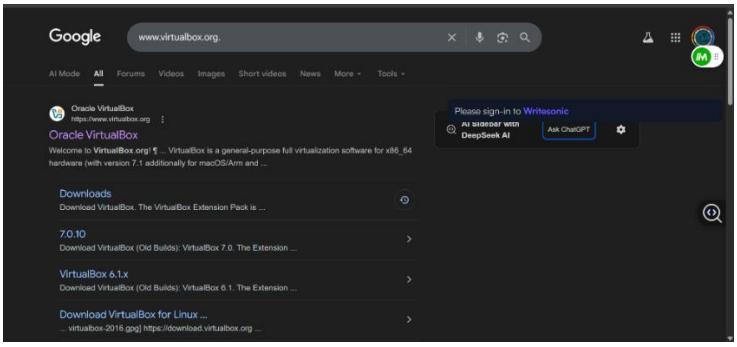
Objective:

Download and install Oracle VirtualBox, then obtain a Linux distribution ISO.

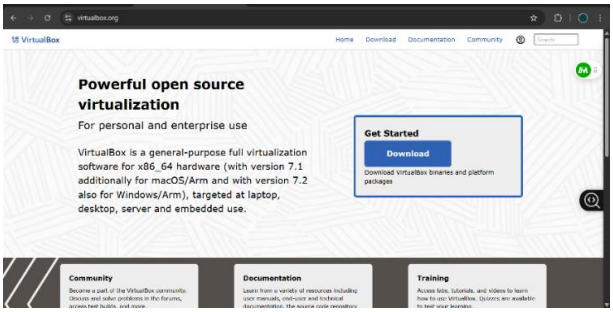
Procedure:

Download VirtualBox

- Go to www.virtualbox.org.



- Click "Download" and choose the installer for your host OS (Windows, Linux, or macOS).

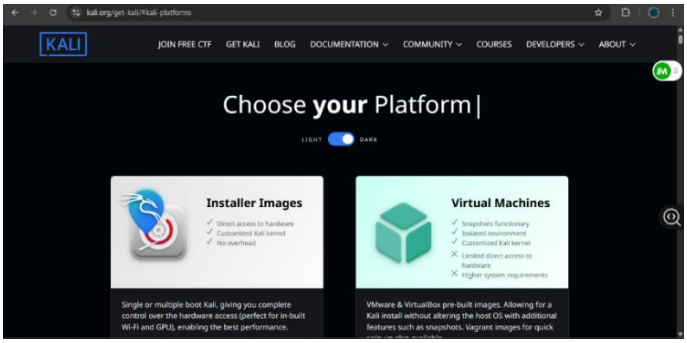




- Run the installer with default settings.

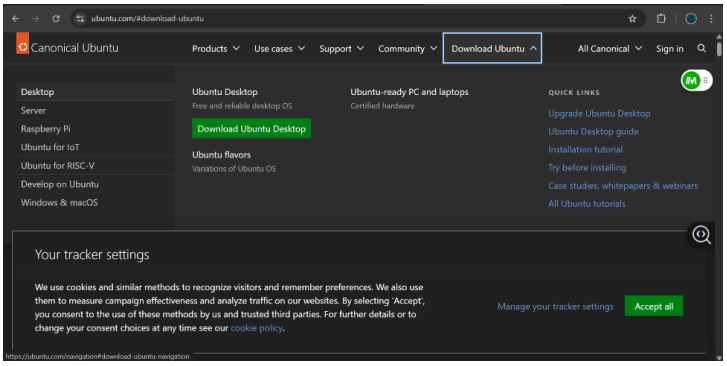
Download an operating system image

- Visit www.kali.org and download the Kali Linux ISO (or a pre-built VirtualBox image).





- Also, download the Ubuntu Desktop ISO from ubuntu.com if needed.



Question & Answer Session:

Scenario:

A student installed VirtualBox on a laptop with 8 GB RAM. She created a VM and allocated 6 GB RAM to it. The host became extremely slow.

i) Why did this happen?

Answer:

The host OS was left with only 2 GB RAM, which is insufficient for smooth operation (Windows 10/11 typically needs 4 GB or more). The host started using the page file (hard disk as "virtual memory"), which is hundreds of times slower than RAM, causing overall sluggishness.

ii) How can she fix it?

Answer:

Reduce the VM's RAM allocation to 2-3 GB, leaving at least 4-5 GB for the host. If she frequently runs VMs, upgrading physical RAM to 16 GB is the best long-term solution.

LAB Task 4

VM Creation and Configuration Basics

Objective:

Understand CPU virtualization and create a bootable USB drive for Ubuntu.

Q1. Explain CPU virtualization in your own words.

CPU virtualization allows a single physical processor to act as multiple logical processors (vCPUs). The hypervisor schedules time slices of the real CPU cores among different VMs. Each VM believes it has its own dedicated CPU.

Benefits:

- Run multiple operating systems concurrently.
- Better use of CPU resources - less idle time.
- Supports cloud and data centre environments.

Drawbacks:

- Very slight performance overhead (usually less than 5% on modern hardware).
- Over-allocating vCPUs can lead to contention and slowdowns.

Example:

Your laptop has 4 CPU cores. You can create a VM with 2 vCPUs and still have 2 cores available for the host. The hypervisor rapidly switches the VM's tasks between the physical cores.

Q2. Scenario:

A user created a Windows 10 VM with only 1 GB RAM. It runs, but is extremely laggy.

i) Why is it slow?

Answer:

Windows 10 64-bit requires at least 2 GB of RAM just to boot, and 4 GB is recommended for basic usability. With 1 GB, the OS constantly swaps data between RAM and the virtual hard disk, causing high disk I/O and CPU usage.

ii) What should the user do?

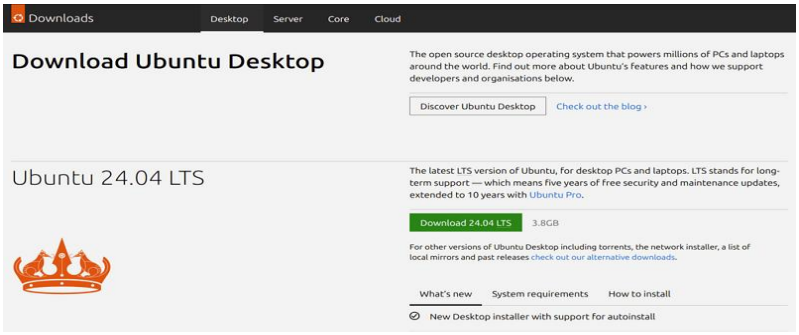
Answer:

Increase the VM's RAM to 4 GB (or more). Also ensure that the VM has at least 2 CPU cores and that Guest Additions are installed for better performance.

Q3. Create a bootable Ubuntu USB.

Steps:

- Download the Ubuntu ISO from the official website.



- Insert a USB drive (minimum 12 GB).
- Use Balena Etcher (or Rufus) to write the ISO to the USB.

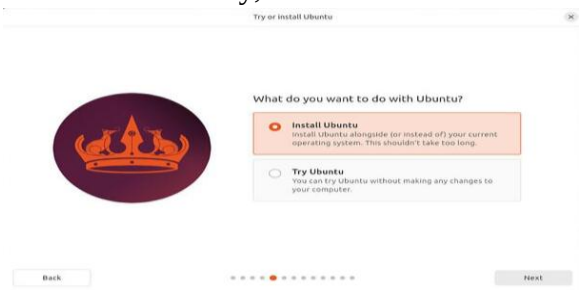


Boot your computer from the USB

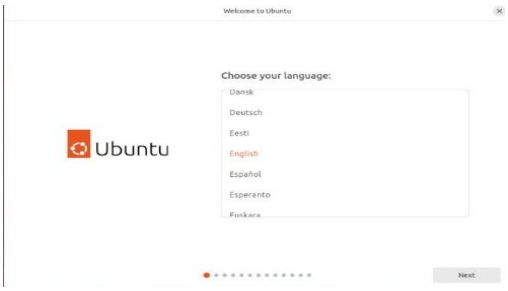
- Insert the bootable USB drive into your computer.
- Restart the computer and access the boot menu (commonly by pressing F12, Esc, F2, or Del during startup).
- Select the USB drive from the boot options to start the Ubuntu installer.

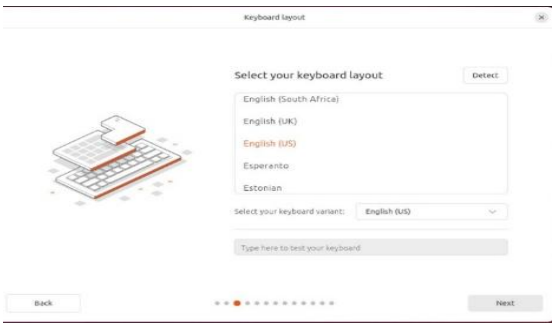
Start the Ubuntu Installation:

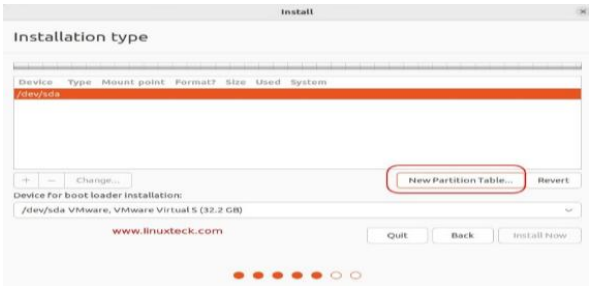
- Once Ubuntu boots, select Try Ubuntu to ensure hardware compatibility.
- When ready, double-click the Install Ubuntu icon on the desktop to begin the installation.



- Follow the Ubuntu installer: choose language, keyboard layout, installation type (e.g., “Install alongside existing OS” or “Erase disk”).







If manual partitioning, create:

- Root partition / (ext4, ≥20 GB)
- Swap partition (size = RAM, optional)
- Home partition /home (remaining space)
- Complete installation, remove USB, and reboot.

Begin Installation

- Review your partition settings and click Install Now.
- Confirm any prompts about writing changes to the disk.

Complete Installation

- Allow the installation process to finish.
- Once done, remove the USB drive when prompted and click Restart Now.



Boot into Ubuntu or Windows:

- Upon rebooting, you'll see the GRUB boot menu, allowing you to select either Ubuntu or Windows to boot into.



LAB Task 5

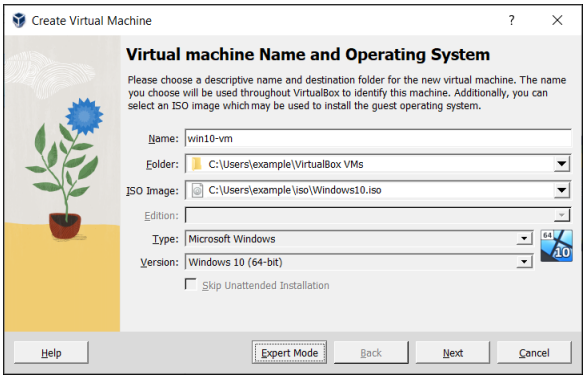
Guest OS Installation Process

Objective:

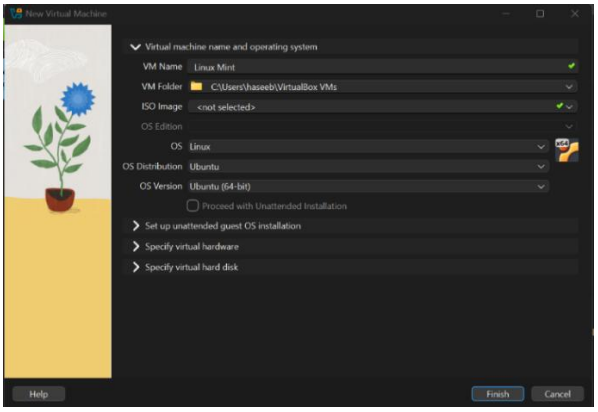
Create a VM in VirtualBox and install a Linux distribution (e.g., Linux Mint).

Step-by-step:

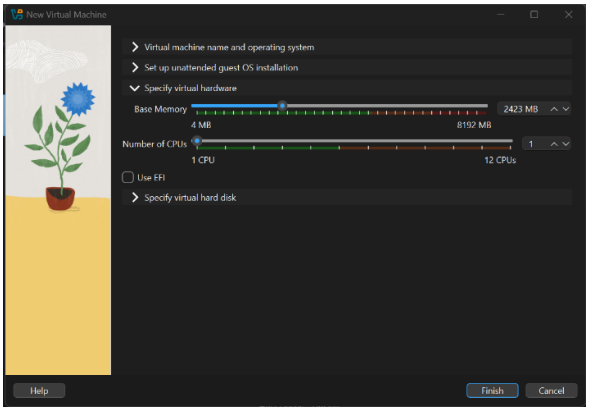
- Open VirtualBox and click New.



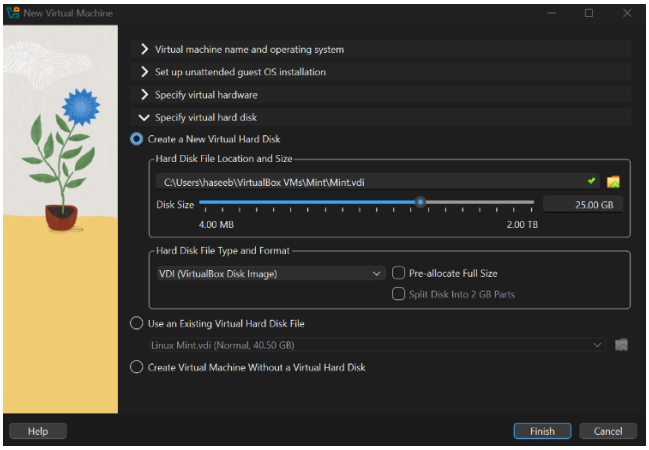
- Name your VM (e.g., “Linux Mint”), select Type: Linux, Version: Ubuntu (64-bit).



- Assign memory (RAM): at least 2 GB, but 4 GB recommended.

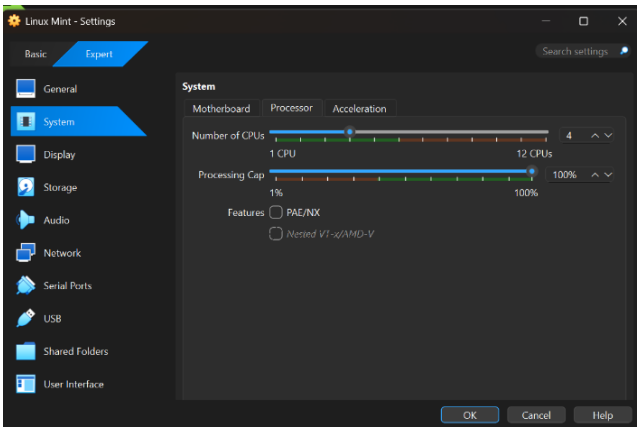


- Create a virtual hard disk: choose VDI (VirtualBox Disk Image), dynamically allocated, size 20-30 GB.

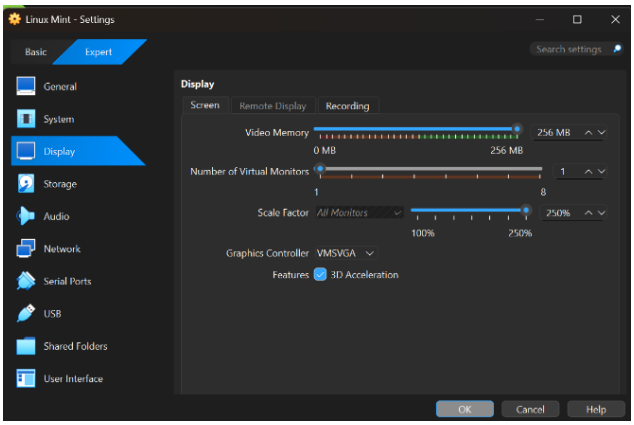


After creation, go to Settings:

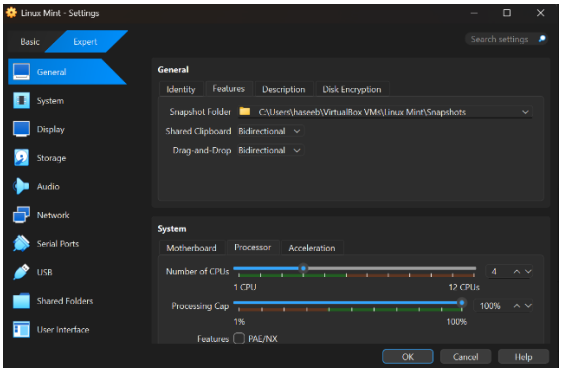
- System → Processor → allocate 4 CPU cores.



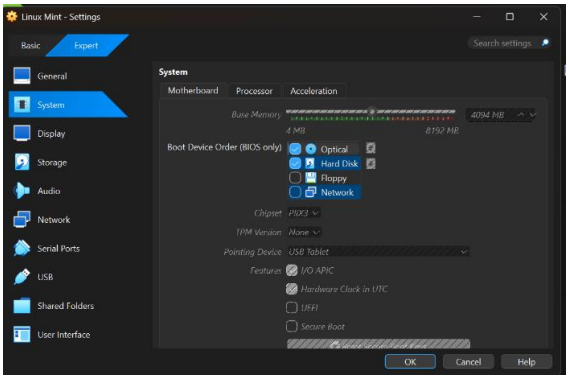
- Display → enable 3D Acceleration, set Video Memory to 256 MB.



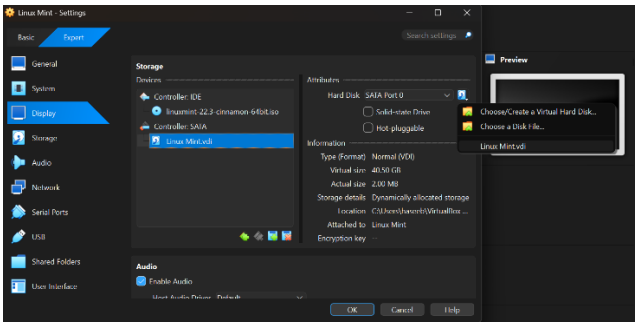
- In General, go to Feature, Shared Clipboard & Drag and Drop → both set to Bidirectional.



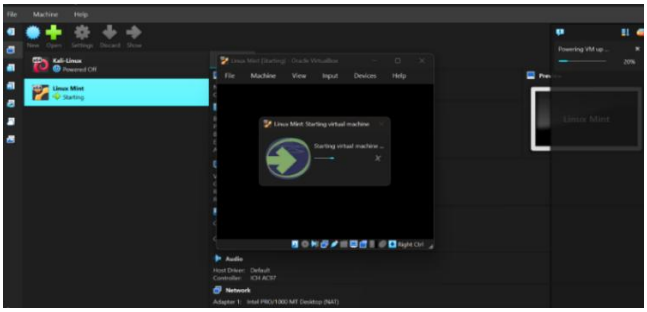
- In System settings, disable the Floppy option and go to the Processor.



- In Storage, load the Linux Mint ISO into the optical drive.



- Start the VM and follow the installer's prompts (language, time zone, user account).

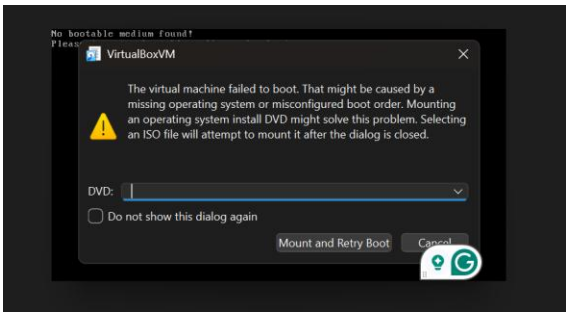


=>Common VirtualBox Troubleshooting & Solutions

When setting up Linux Mint in Oracle VirtualBox, users often encounter boot errors or display scaling issues. Below are the standard procedures to resolve these.

1. Issue: Virtual Machine Failed to Boot (No Bootable Medium Found)

Symptoms: Upon starting the VM, a black screen or a pop-up appears stating: "The virtual machine failed to boot. A missing operating system might cause that..."



Cause: This happens when the Virtual Machine cannot locate the Linux Mint ISO file to begin the installation process.

Step-by-Step Solution:

- Close the error dialog and Power Off the virtual machine.
- Select your VM and click on the Settings (gear icon).
- Navigate to the Storage section on the left sidebar.
- Under the Storage Devices tree, look for the Controller: IDE and click on the Empty disk icon.
- On the right side, click the Optical Drive (Blue Disc icon) and select "Choose a disk file..."
- Locate and select your Linux Mint ISO file from your local storage.
- Click OK and start the VM again.

2. Issue: Incorrect Screen Size or Display Scaling

Symptoms: The guest OS window is either too small to read or the resolution does not fit your monitor properly, making the interface difficult to navigate.

Cause: Mismatched resolution settings between the Host OS and the Guest VM.

Step-by-Step Solution:

Go to the VM Settings and select the Display tab.

Adjust Scale Factor: Under the "Screen" tab, locate the Scale Factor slider.

- If the text/icons are too small, increase the scale to 150% or 200%.
- This forces the VM to enlarge the display output regardless of the internal resolution.

Video Memory: Increase the Video Memory slider to its maximum (usually 128MB or 256MB). This ensures the UI animations (like Mint's Cinnamon desklets) run smoothly

Graphics Controller: Ensure the Graphics Controller is set to VMSVGA for the best compatibility with Linux Mint.

3D Acceleration: Check the box for Enable 3D Acceleration to improve window rendering speed.

Q&A after installation:

Scenario:

After installing Ubuntu in a VM, the screen resolution is stuck at 800×600.

i) What is the likely cause?

Answer:

The VM is using a basic VESA graphics driver because Guest Additions are not installed.

ii) How to fix it?

Answer:

- In VirtualBox, go to Devices → Insert Guest Additions CD image.

Inside the VM, open a terminal and run:

```
sudo apt update

sudo apt install build-essential dkms linux-headers-$(uname -r)

sudo mount /dev/cdrom /mnt

cd /mnt

sudo ./VBoxLinuxAdditions.run
```

- Reboot the VM. Also ensure 3D Acceleration is enabled in VM settings.

Q3. What is EFI/UEFI and when should it be used?

Answer:

EFI (Extensible Firmware Interface) / UEFI is modern firmware that replaces the old BIOS. It supports large disks (over 2 TB), faster booting, Secure Boot, and GPT partition tables.

Enable EFI when:

- Installing Windows 10/11 or recent Linux distributions.
- Using a disk with GPT partitioning.
- Needing Secure Boot for security.
- Running a VM that requires UEFI (some Linux distros).

Disable EFI (use Legacy BIOS) when:

- Installing older OS like Windows 7 in MBR mode.
- The VM requires legacy compatibility.

LAB Task No 6

Understanding and Managing Virtual Hard Disk Resources

Objective:

Use Linux command-line tools to inspect disk usage, partition layout, storage space, memory, and running processes.

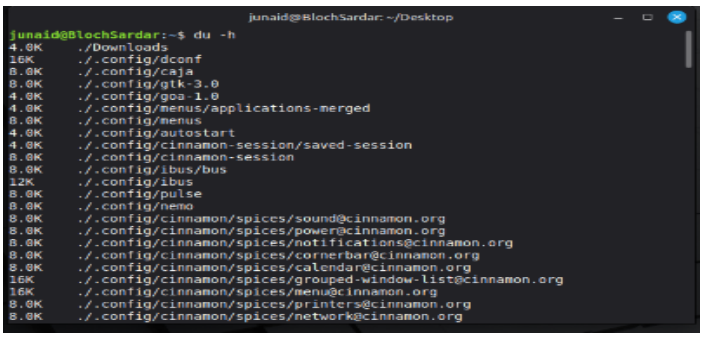
Why this matters:

A virtual machine has limited virtual hard disk space and RAM. Knowing how to check what is using space and memory helps you avoid crashes and slowdowns.

1. Check Disk Usage of Files and Folders

Command:

- du –h



What it does:

Shows the size of every file and subfolder inside the current directory.

du = disk usage

-h = human-readable (KB, MB, GB)

Example output (conceptual):

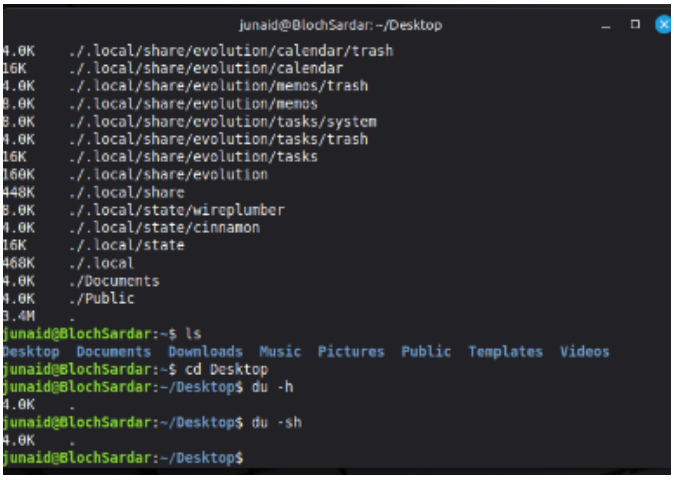
- 4.0K ./Desktop
- 12M ./Downloads
- 342M ./Documents

Meaning of units:

- K → Kilobytes (roughly 1,000 bytes)
- M → Megabytes (1,000 KB)
- G → Gigabytes (1,000 MB)

To see only the total size of a specific folder (e.g., /Desktop):

- du -sh /Desktop or du –h /Desktop



-s = summary (only the total, not each subfolder)

Example output:

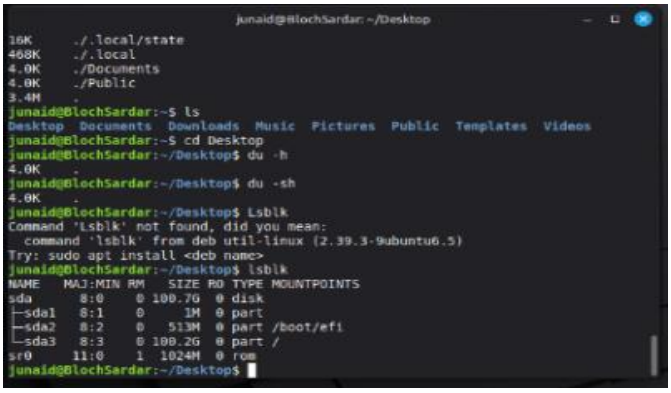
- 4.0K /Desktop

→The entire /Desktop older (all user files) uses 4.0K.

2. Examine Disk Partitions

Command:

- Lsblk



(Short for “list block devices”)

What it shows:

- **NAME** - Partition or disk name (e.g., sda1, sda2)
- **SIZE** - Size of that partition
- **TYPE** - disk (physical/virtual disk) or part (partition)
- **MOUNTPOINT** - Where the system has attached that partition (e.g., / for root, /home for user data, [SWAP] for swap space)

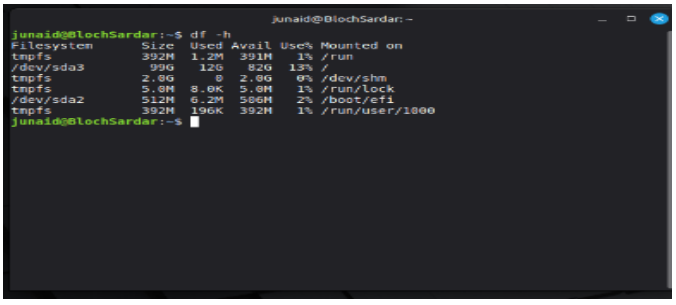
Example:

NAME	SIZE	MOUNTPOINT
sda	40G	
├─sda1	20G	/
├─sda2	2G	[SWAP]
└─sda3	18G	/home

3. Check Overall Disk Space Usage (Mounted File Systems)

Command:

- df –h



(df = disk free)

What each column means:

Column	Meaning
File system	Partition name (e.g., /dev/sda1)
Size	Total capacity
Used	Space already occupied
Avail	Free space available for use
Use%	Percentage of used space
Mounted on	Directory where this filesystem is attached (e.g., /)

Example:

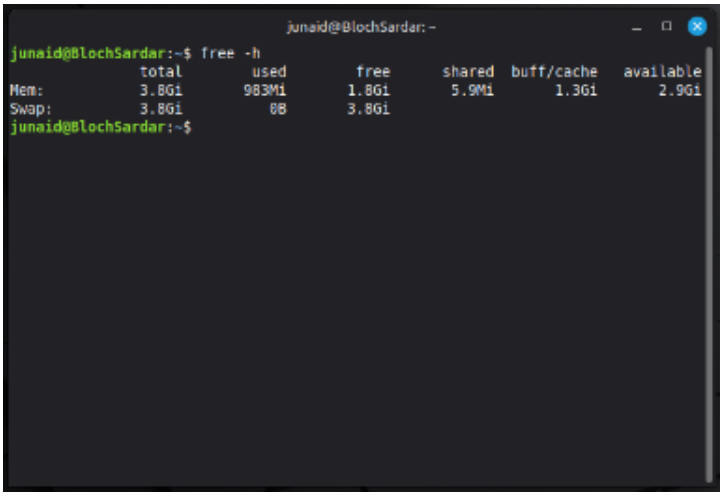
File system	Size	Used	Avail	Use% Mounted on
/dev/sda1	20G	12G	8G	60% /

→ Root partition is 60% full, with 8 GB free.

4. Check Memory (RAM) and Swap Usage

Command:

- free –h



(free = show free and used memory; -h = human units)

Columns explained:

Column	Meaning
total	Total physical RAM installed
used	RAM currently in use by programs and system
free	Completely unused RAM
shared	Memory used by multiple processes (e.g., shared libraries)
buff/cache	Data cached from disk to speed up access – can be freed if needed
available	Realistic estimate of RAM available for new applications

Swap:

When RAM becomes full, the system moves less-used data to swap (a special partition or file on disk). Swap is much slower than RAM.

Example output:

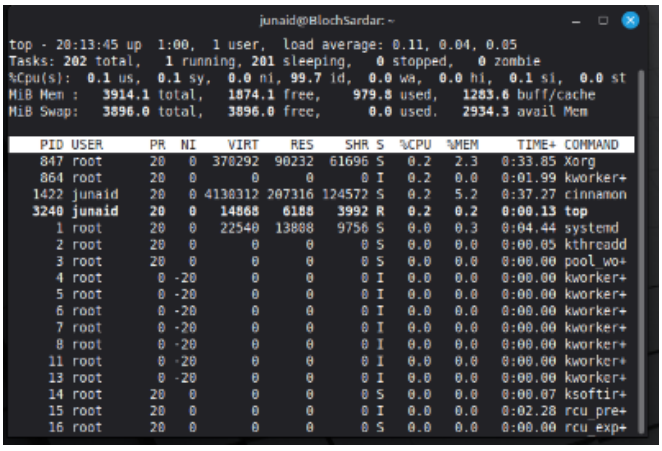
	total	used	free	shared	buff/cache	available
Mem:	7.7G	2.1G	3.2G	150M	2.4G	5.1G
Swap:	2.0G	0.0B	2.0G			

→7.7 GB total RAM, 2.1 GB used, 5.1 GB available for new programs.

5. Live Process and CPU Monitoring – top

Command:

- top



→Press q to quit.

What you see:

Header area:

- **load average** - System workload over 1, 5, and 15 minutes. Lower is better (ideally below number of CPU cores).
- **Tasks -** How many processes are running, sleeping, stopped.
- **%Cpu(s)** - Breakdown:
- **us =** user processes (your programs)
- **sy** = system (kernel) work
- **id** = idle (free CPU)
- **Mem and Swap** - current usage.

Process table:

- **PID** - Process ID (used to kill a process)
- **USER** - Who owns the process
- **%CPU** - CPU usage percentage
- **%MEM** - RAM usage percentage
- **COMMAND** - Name of the program

Useful keys inside top:

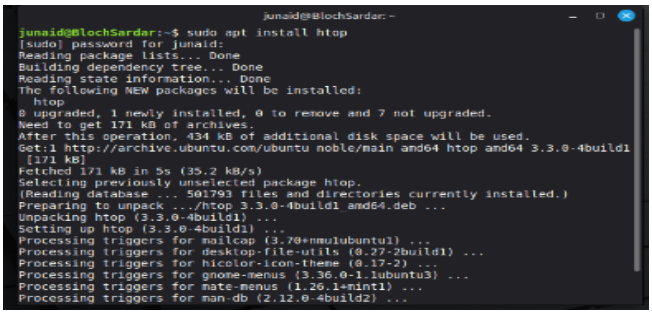
- **P** → sort by CPU usage
- **M** → sort by memory usage
- **k** → kill a process (you will be asked for PID)
- **q** → quit

6. Better Interactive Monitor - htop

htop is an improved version of top with colours, mouse support, and easier navigation.

Install it (if not already present):

- sudo apt install htop



Run it:

- htop



Features:

- Coloured bars for CPU, RAM, and swap usage.
- Scroll up/down with arrow keys.
- Kill a process by selecting it and pressing F9.
- Quit with F10 or q.

Example:

The top section shows each CPU core's usage as a bar; the bottom section lists processes.

LAB Task No 7

Checking Resource Allocation and Stress Testing the VM

Objective:

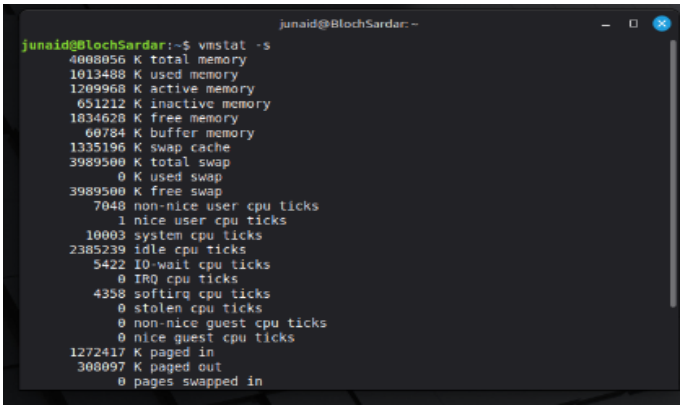
Learn commands to see how many CPU cores and how much RAM your VM has been given, then deliberately create a high load to test stability.

Part A – View Allocated Resources

Command	What it reveals
lscpu	CPU architecture (x86_64), number of cores, threads per core, model name, virtualization support
free -h	Total, used, and free RAM (as seen inside the VM)
nproc	Number of CPU cores assigned to this VM
grep MemTotal /proc/meminfo	Total RAM allocated to the VM (in kilobytes)
uname -a	Kernel name, hostname, kernel release, machine hardware
hostnamectl	OS name, version, architecture, and hardware details (if systemd is used)
vmstat	One-line summary of processes, memory, swap, I/O, system, and CPU
iostat	Disk read/write and CPU statistics (install sysstat if missing)

Example of checking System hardware summary:

- vmstat –s



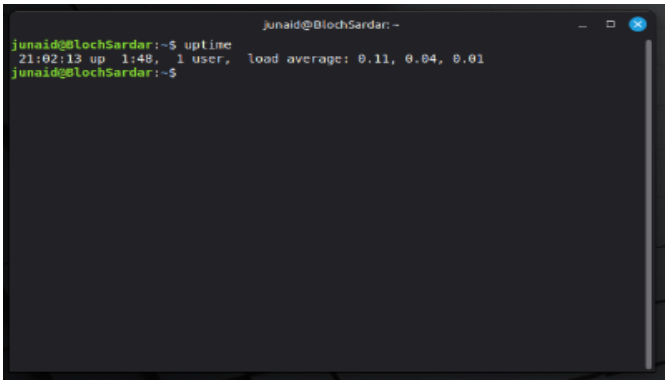
Part B – Real-Time Monitoring

For quick terminal monitoring:

- **top** - built-in, works everywhere.
- **htop** - recommended for daily use (colourful, mouse-friendly).
- **vmstat 1** - updates every second, shows CPU and memory changes over time.
- **iostat** - good for seeing disk activity and CPU idle time.

Check system load average (1, 5, 15 minutes):

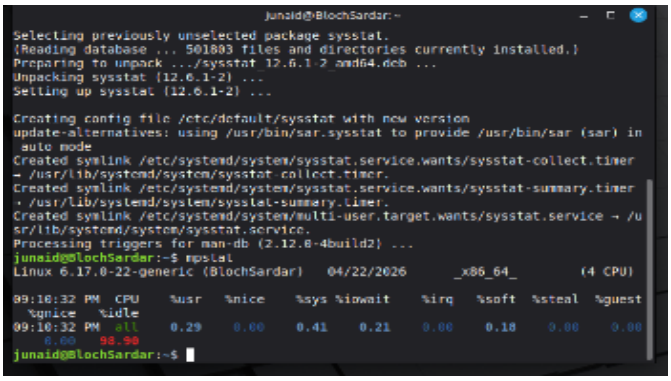
- uptime



A load average below the number of CPU cores is good.

Monitor only CPU:

- mpstat

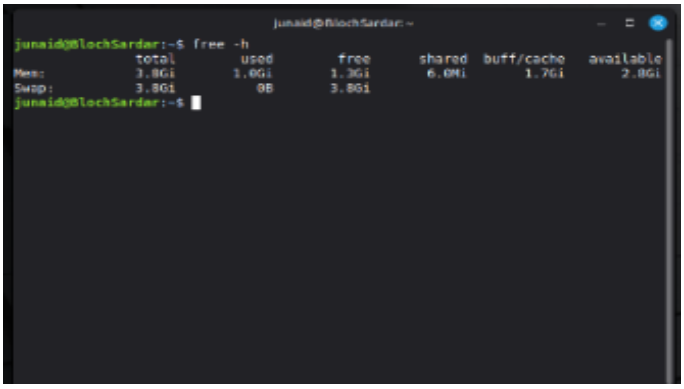


If mpstat is not found, then install it through the command:

- Sudo apt install sysstat

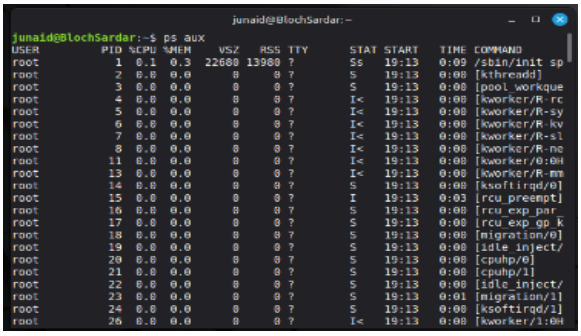
Only memory:

- free –h



List all running processes with details:

- ps aux



GUI option (if your VM has a desktop):

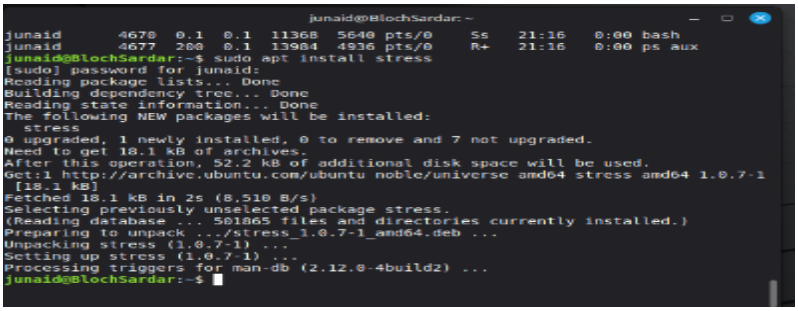
Search for “System Monitor” (GNOME/KDE) – shows graphs of CPU, memory, and network.

Part C - Stress Testing the Virtual Machine

→**Warning:** Stress tests push CPU and RAM to 100% usage. The VM may become slow, hot, or unresponsive. Close important work before starting.

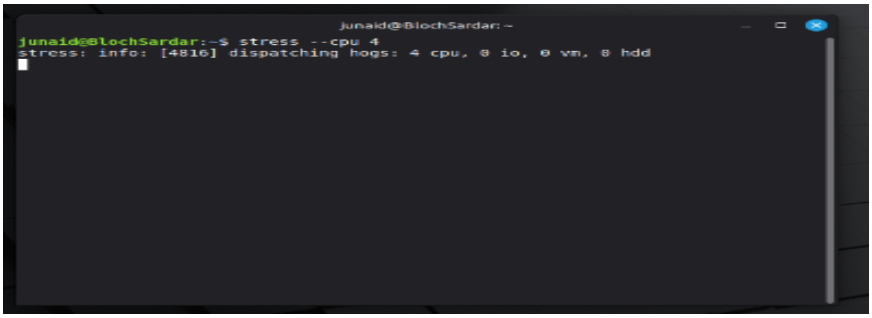
1. Install the stress tool

- sudo apt install stress



2. Stress only the CPU

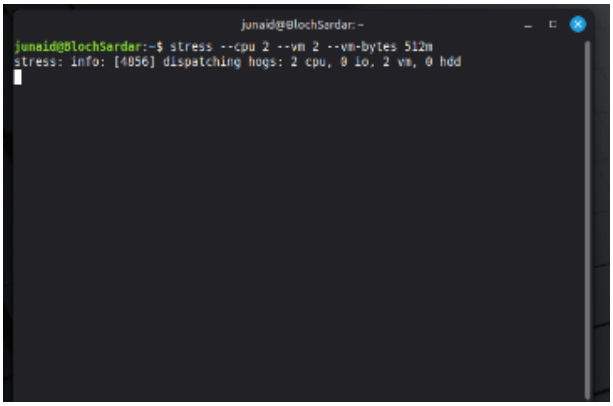
- stress --cpu 4



Replace 4 with the number of CPU cores your VM has (check with nproc).

3. Stress both CPU and RAM

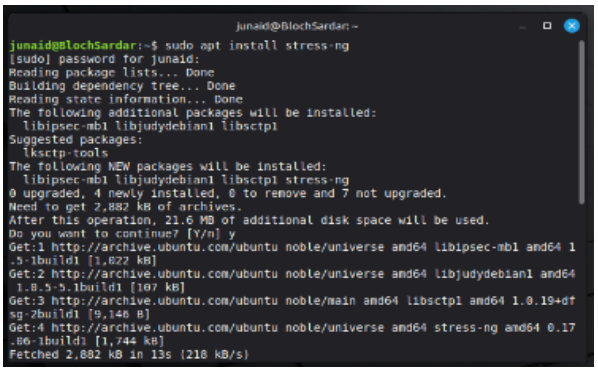
- stress --cpu 2 --vm 2 --vm-bytes 512M



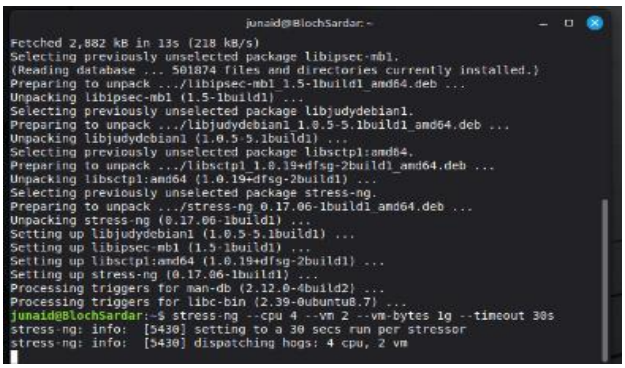
- **--cpu 2** → use 2 CPU cores
- **--vm 2** → start 2 memory-eating workers
- **--vm-bytes 512M** → each worker allocates 512 MB of RAM

4. Advanced tool – stress-ng (more control)

- sudo apt install stress-ng



- stress-ng --cpu 4 --vm 2 --vm-bytes 1G --timeout 30s



The --timeout 30s stops the test automatically after 30 seconds – safer.

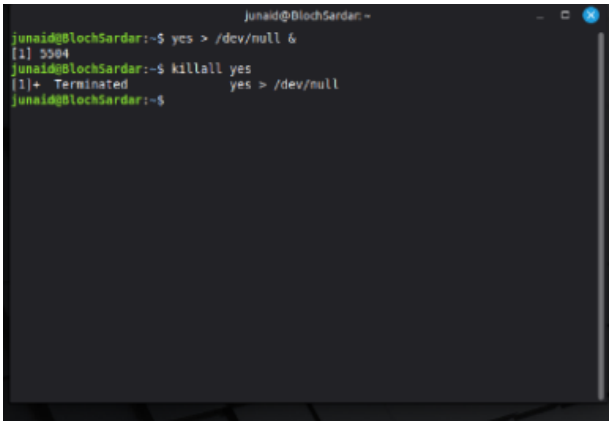
5. Simple infinite CPU load (no extra installation)

- yes > /dev/null &



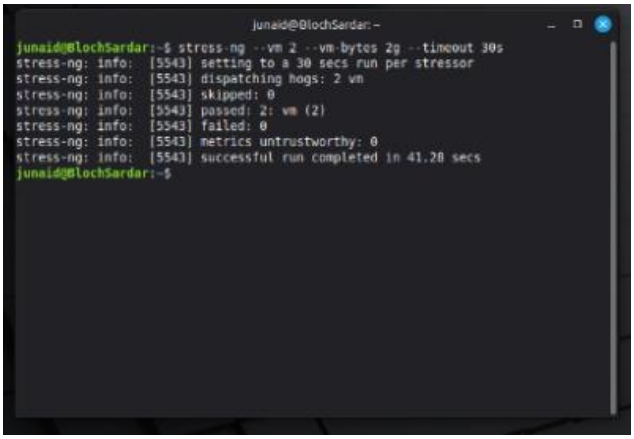
Run this command several times to increase load. Stop all with:

- killall yes



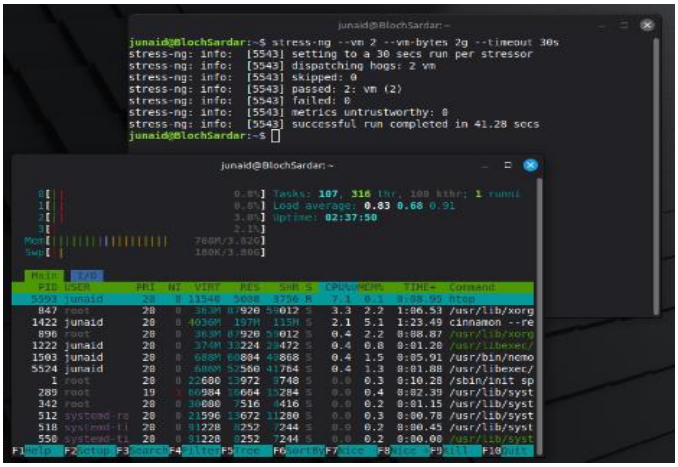
6. Stress only RAM

- stress-ng --vm 2 --vm-bytes 2G --timeout 30s



Monitor while stressing:

Open a second terminal and run htop. You will see CPU bars turn red (100% usage).



Stop stress test:

- For stress: killall stress
- For stress-ng with --timeout: it stops automatically.

LAB Task No 8

Snapshots, System Modification, and Restoration

Objective:

Save a complete state of your VM (snapshot), make changes, and then revert to the saved state.

1. What is a Snapshot?

A snapshot is like a “freeze-frame” of your VM’s entire state – including the disk contents, RAM, and settings. If something goes wrong later (e.g., after installing broken software or changing critical settings), you can restore the snapshot and the VM will be exactly as it was at that moment.

Snapshots – Freezing the Entire VM

A snapshot captures the complete state of a VM at one moment. This includes:

- The contents of virtual RAM (every byte of memory).
- The state of virtual CPU (registers, program counter, etc.).
- The state of all virtual devices (network connections, USB, etc.).
- The exact power status (running, paused, or off).

When you take a snapshot, the hypervisor pauses the VM briefly, saves all this information to a special file on disk, then resumes the VM. Later, if you restore that snapshot, the hypervisor reads the saved state and puts the VM back exactly as it was – as if time had rewound. Any changes made after the snapshot are lost.

This is extremely useful for testing, recovery, and creating safe checkpoints before risky operations.

A Simple Analogy

Think of a snapshot like a save point in a video game:

- You save the game at a safe spot.
- Then you run into a dangerous area, fight monsters, and may die.
- You load the save point - you are back at the safe spot, as if nothing bad ever happened.

Snapshots for VMs work the same way, but they save everything - the entire computer state.

2. Creating a Snapshot in VirtualBox (for Linux Mint)

Steps:

- Open VirtualBox. (You can leave the VM running, but shutting it down first is safer and faster.)
- Select your Linux Mint VM.
- Click the Snapshots tab (next to Details).
- Click the Take Snapshot icon (camera with a + sign) or press Host + T (the Host key is usually the Right Ctrl key).

Give it a clear name and description:

- **Name:** Before_System_Update (or any meaningful name)
- **Description:** Clean Linux Mint install, no extra tools
- **Click OK**.

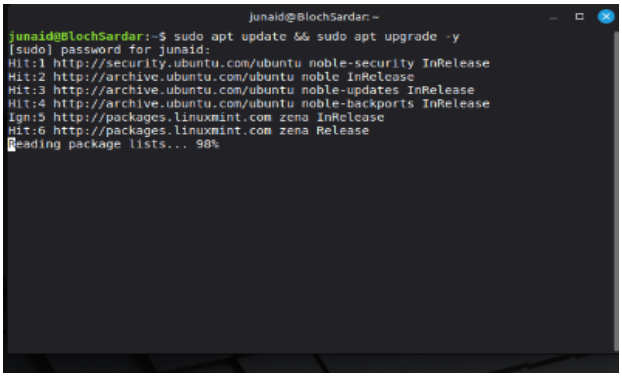
Result: The snapshot is saved. You can now modify your Linux Mint VM without worry.

3. Modifying the System (After Taking a Snapshot)

Make any changes you like – for example:

Update all packages (Linux Mint uses apt):

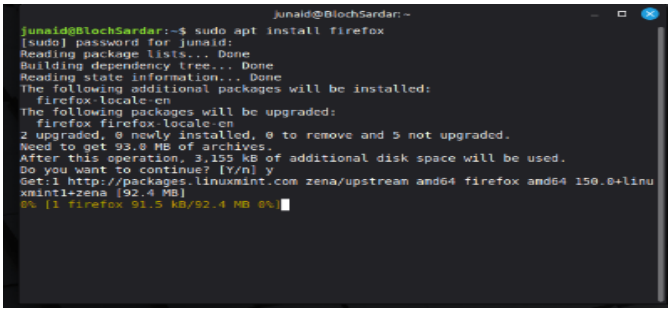
- sudo apt update && sudo apt upgrade –y



It takes time.

Install new software (e.g., Firefox if not already present):

- sudo apt install firefox

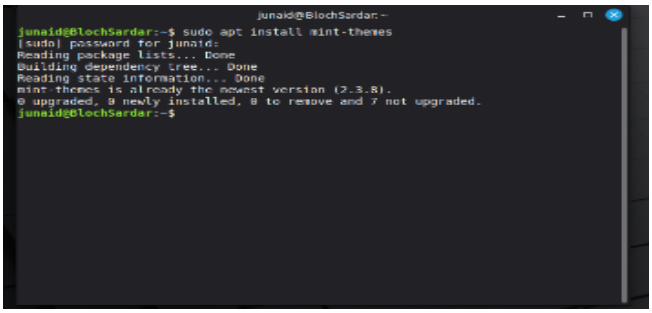


It also takes time.

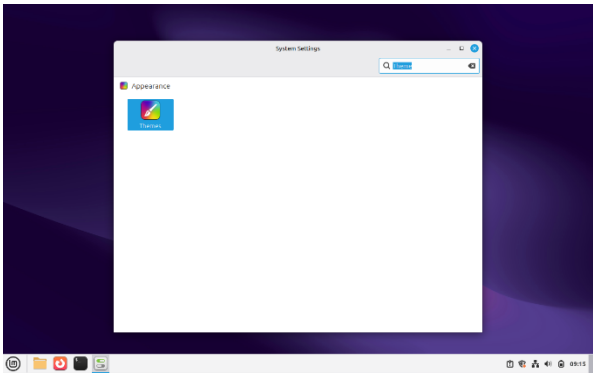
Change desktop appearance (themes, icons, fonts) in Linux Mint:

Linux Mint (Cinnamon edition) comes with a built-in “Themes” settings application. You can also install additional themes from the command line:

- sudo apt install mint-themes

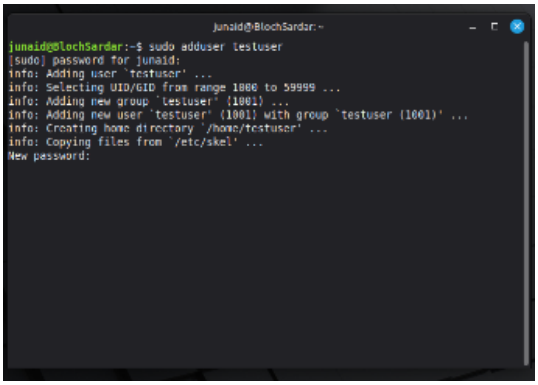


Then open System Settings →Themes to apply.



Add a new user:

- sudo adduser testuser

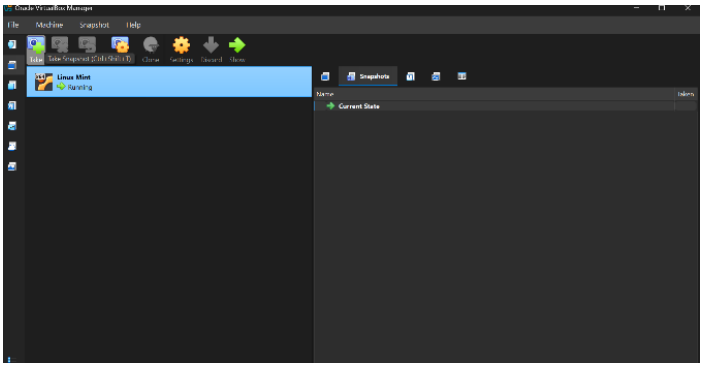


For deleting only additional user:

- Sudo deluser username

4. Restoring a Snapshot

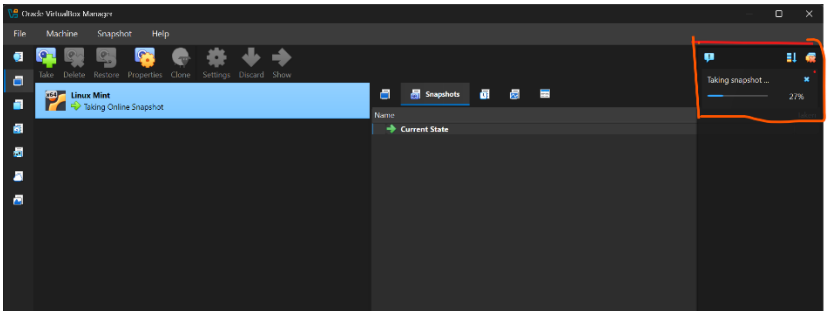
In VirtualBox:



- Power off the VM (recommended – restoring a running VM can sometimes cause issues).

- Go to the Snapshots tab.
- Select the snapshot you want to revert to.
- Click Restore (button at the top).

Taking online snapshot:



You may see two options:

- Restore only discards all changes made after that snapshot.
- Restore and create another snapshot to keep the current state as a new snapshot before reverting (useful if you are not sure).

In VMware Workstation (if used):

VM → Snapshot → Snapshot Manager → select snapshot → Go To.

Quick restore in VirtualBox: Right-click the VM → Restore Snapshot.

What happens after restore?

- All files, settings, and installed programs that were added after the snapshot are gone.
- The VM becomes an exact copy of the moment the snapshot was taken.

5. Best Practices for Snapshots

Name snapshots clearly - e.g., clean_install, before_network_config, after_updates_2026_04_22.

Take snapshots before:

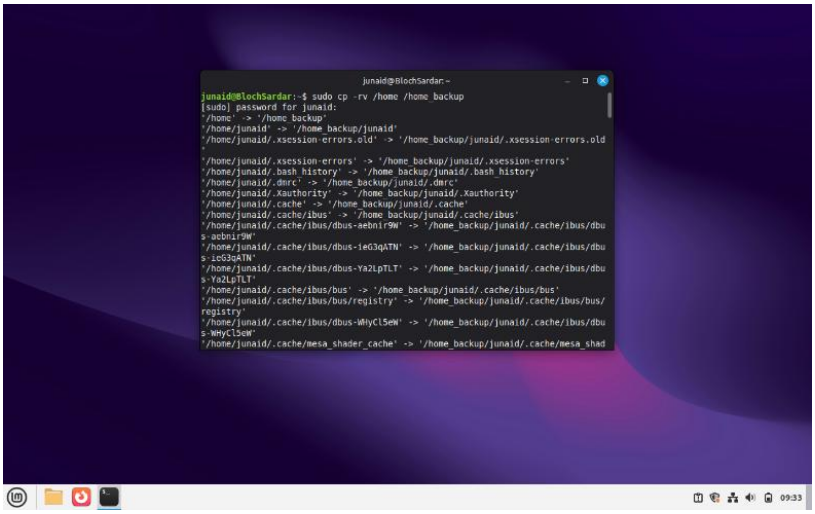
- Major system updates (apt upgrade)
- Installing unfamiliar or untrusted tools
- Changing network settings (NAT ↔ Bridged)
- Running stress tests or security experiments

Snapshots use disk space - each snapshot stores the differences from the previous state. Delete old snapshots you no longer need (VirtualBox → Snapshots, right-click → Delete).

Do not keep dozens of snapshots - too many snapshots can slow down your VM and increase disk usage. Keep only the essential ones.

Alternative quick backup (file level - less complete than a snapshot):

- sudo cp -rv /home /home_backup



This only backs up user data, not system files or settings. A full VM snapshot is better for complete recovery.

LAB Task No 9

NAT (Network Address Translation) – Safe Internet for Your VM

Objective:

Configure your Linux Mint VM to use NAT mode, allowing internet access while keeping it hidden from the outside network.

What is NAT?

- NAT (Network Address Translation) is the default network mode in VirtualBox and VMware.
- The VM gets a private IP address (usually 10.0.2.15).
- The host computer acts like a router:
- The VM sends all internet traffic through the host's physical network.
- The host changes the source address to its own IP before sending it out.

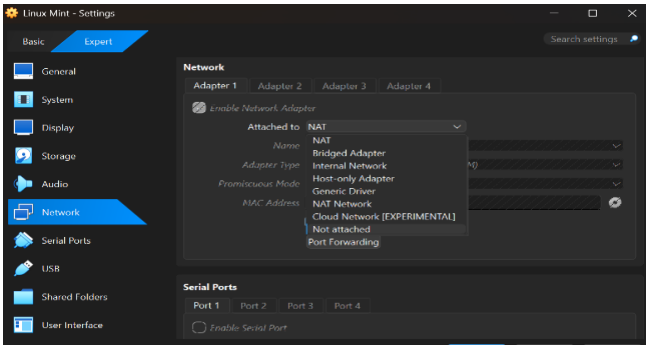
Result: Your Linux Mint VM can browse the web, download updates, and install software using apt, but devices on your real local network cannot see or connect to the VM.

Think of it as: The VM is inside a "private room" and the host is the front door.

1. Setting Up NAT in VirtualBox for Linux Mint

Open VirtualBox → select your Linux Mint VM → Settings → Network.

- Make sure Adapter 1 is enabled.
- From the Attached to: dropdown, choose NAT.
- Click OK.



In VMware Workstation (if used):

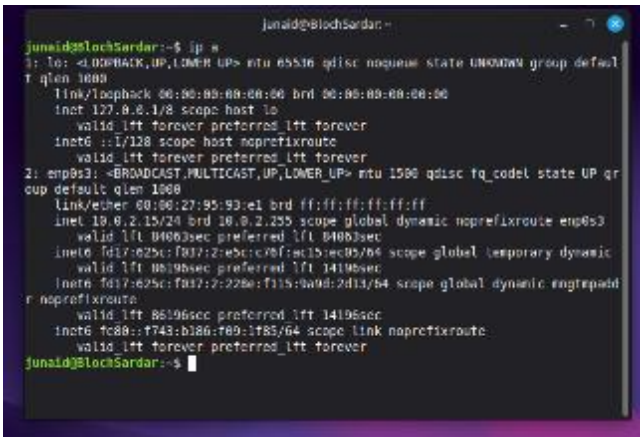
VM → Settings → Network Adapter → select NAT (Shared with host).

2. Verifying NAT is Working (Inside Your Linux Mint VM)

Boot your Linux Mint VM and open a terminal (Ctrl + Alt + T).

Check the assigned IP address:

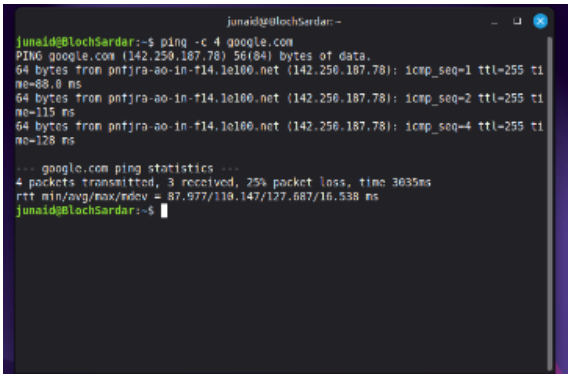
- ip a



You should see an IP like 10.0.2.15 (that's the NAT private range).

Test internet connectivity (Linux Mint can ping google.com):

- ping -c 4 google.com



If you get replies, NAT works. Press Ctrl + C to stop.

Find the default gateway:

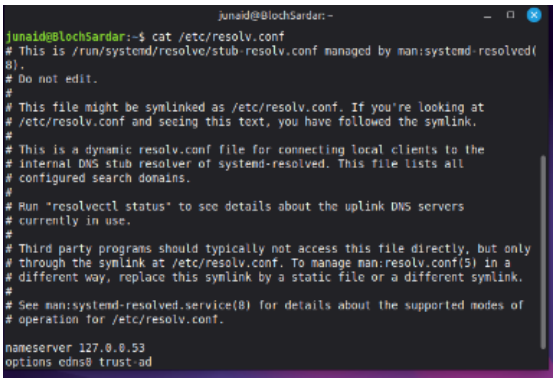
- ip route



Look for a line starting with default via – usually 10.0.2.2.

Check DNS settings:

- cat /etc/resolv.conf



Should contain nameserver 10.0.2.3 (VirtualBox's internal DNS) or a real DNS like 8.8.8.8.

3. NAT vs Bridged – Quick Comparison (for Linux Mint)

Feature	NAT	Bridged
VM's IP address	Private (e.g., 10.0.2.15)	Real LAN IP (e.g., 192.168.1.100)
Can other devices see the VM?	No	Yes
Internet access	Yes (through host)	Yes(direct)
Security	Higher (hidden)	Lower (exposed)

Note for Linux Mint users:

All commands shown (ip a, ping, ip route, cat, systemctl restart NetworkManager) work exactly the same in Linux Mint as in Ubuntu or Kali. No additional configuration is needed.

LAB Task No 10

Bridged Networking - Making the VM a Full Member of Your Local Network

Objective:

Configure bridged mode so your VM gets its own IP address from your router and can communicate directly with other devices on the same network.

What is Bridged Mode?

In bridged mode, the hypervisor (VirtualBox/VMware) connects the VM's virtual network card directly to your physical network card.

Your router's DHCP server gives the VM a real IP address from your local subnet (e.g., 192.168.1.100).

The VM behaves like any other device on your network – it can:

- Ping your phone or printer.
- Run a web server that other devices can access.
- Be scanned by other devices.

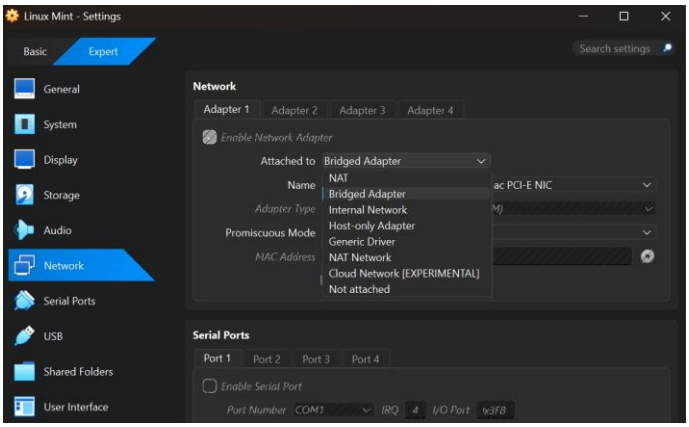
Example:

- Host computer IP: 192.168.1.5
- VM IP: 192.168.1.10
- Both are on the same network and can talk directly.

1. Setting Up Bridged Mode in VirtualBox

Open VirtualBox, select your VM → Settings → Network.

- Adapter 1: Enable Network Adapter.
- Attached to: → Bridged Adapter.



Name: Choose the correct physical interface:

- If you are connected via Wi-Fi, select your wireless adapter (e.g., wlp2s0, Qualcomm Atheros, or "Wi-Fi").
- If you are using Ethernet, select your wired adapter (e.g., eth0, "Realtek PCIe").
- Click OK.

In VMware Workstation:

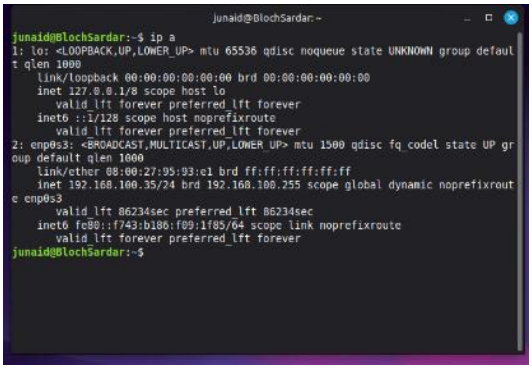
VM → Settings → Network Adapter, select Bridged (Connected directly to physical network).

2. Verifying Bridged Mode Inside the VM

Start the VM and open a terminal.

Check the IP address:

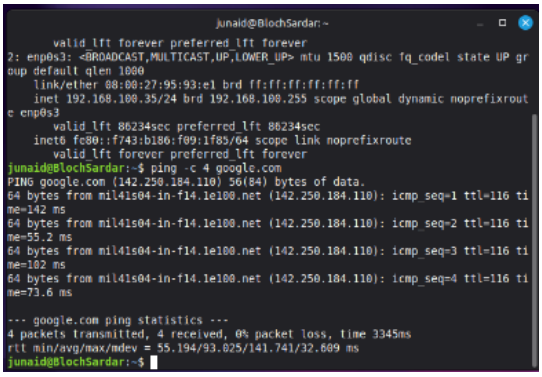
- ip a



You should see an IP in the same range as your host (e.g., 192.168.1.100/24).

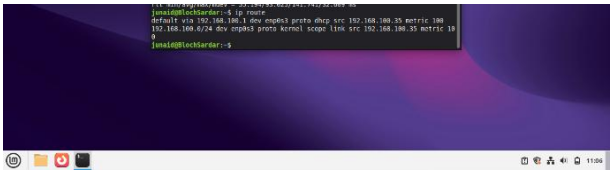
Test internet:

- ping -c 4 google.com



Check the default gateway (should be your router's IP):

- ip route



See other devices on your local network:

- arp -a

or

- ip neigh



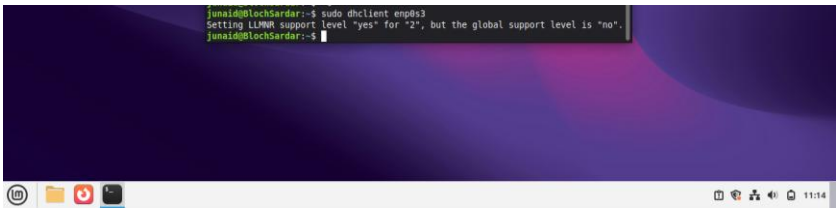
3. Troubleshooting Bridged Mode

Problem 1: VM has no IP address

Solution:

Force the VM to ask for an IP via DHCP:

- sudo dhclient eth0



(Replace eth0 with your VM's interface name, seen in ip a.)

➔ My VM's interface name is "enp0s3"

Problem 2: No internet, even with an IP

- Make sure you selected the correct physical adapter in VirtualBox settings. If you are on Wi-Fi, try switching to Ethernet (or vice versa).

- Some Wi-Fi networks (especially public or corporate) block bridged traffic. In that case, use NAT instead.

Problem 3: Bridged works but Wi-Fi is unreliable

This is a known limitation. Many wireless drivers and routers do not handle bridging well.

Solutions:

- Use an Ethernet cable.
- Switch to NAT mode (internet still works, but VM is hidden).

4. Manual IP Configuration (If DHCP Fails Completely)

You can assign a static IP manually:

Assign an unused IP in your network range (e.g., 192.168.1.200)

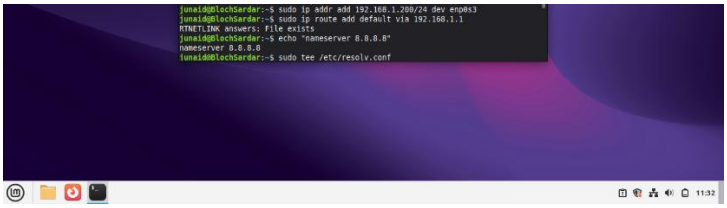
- sudo ip addr add 192.168.1.200/24 dev enp0s3

Set your router's IP as the default gateway

- sudo ip route add default via 192.168.1.1

Set a DNS server (Google's public DNS)

- echo "nameserver 8.8.8.8" | sudo tee /etc/resolv.conf



Note: This is temporary; to make it permanent, you would edit network configuration files (outside the scope of this task).

5. When to Use Bridging Instead of NAT

Choose bridged when you want:

- The VM to be accessible from other devices on your network (e.g., SSH into it from your phone).
- To run network scanning tools like nmap against real devices.
- To practice hacking or networking labs where the VM must have its own IP.
- To test a server (web, FTP, game) inside your local network.
- Choose NAT when you only need internet access and want the VM to be hidden for security.

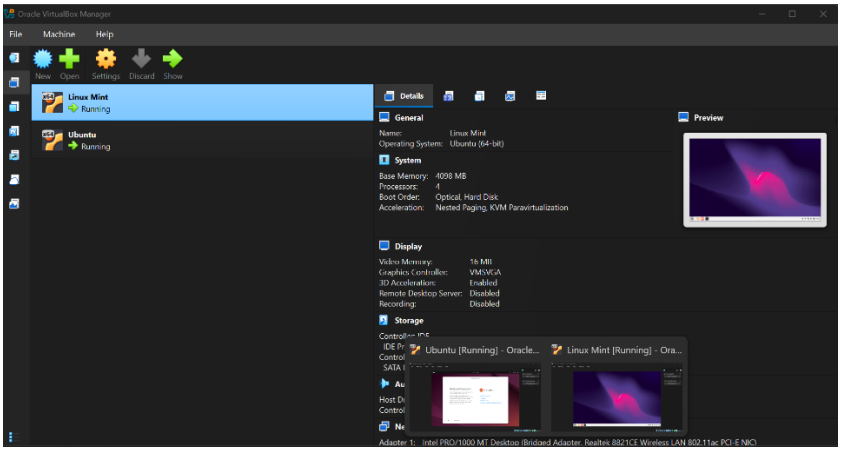
LAB Task 11

Running Two VMs Simultaneously & Observing Resource Distribution

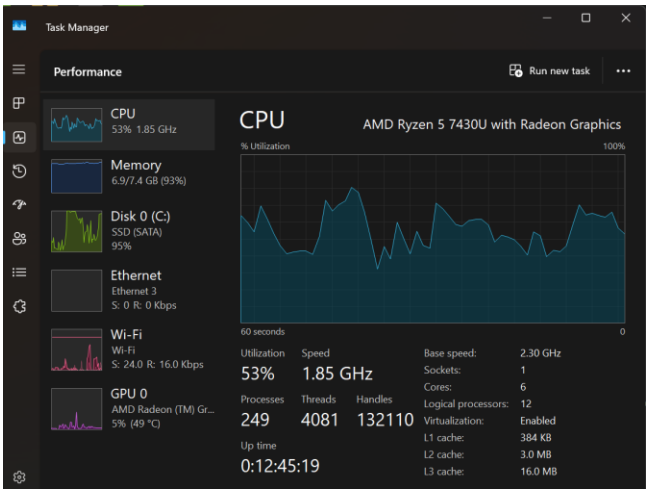
Objective:

- Opened Oracle VirtualBox.
- Started two VMs: Linux Mint and Ubuntu - both running at the same time.
- Opened Task Manager on the host OS (Windows).
- Observed CPU, Memory, and Disk usage.

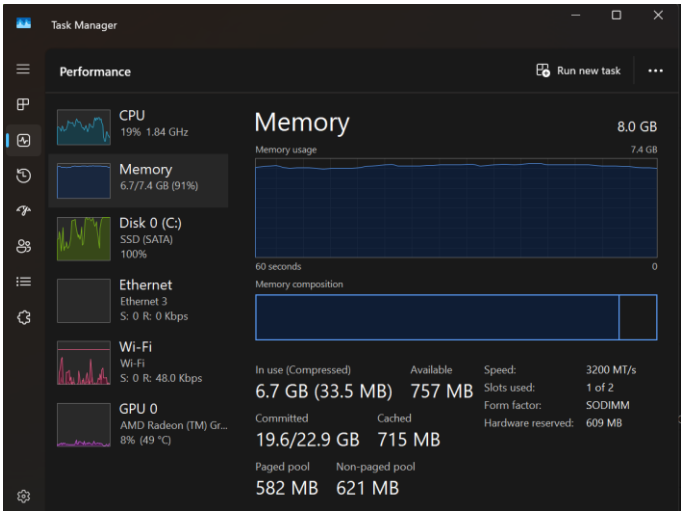
I. VirtualBox manager with both Linux Mint and Ubuntu in Running state.



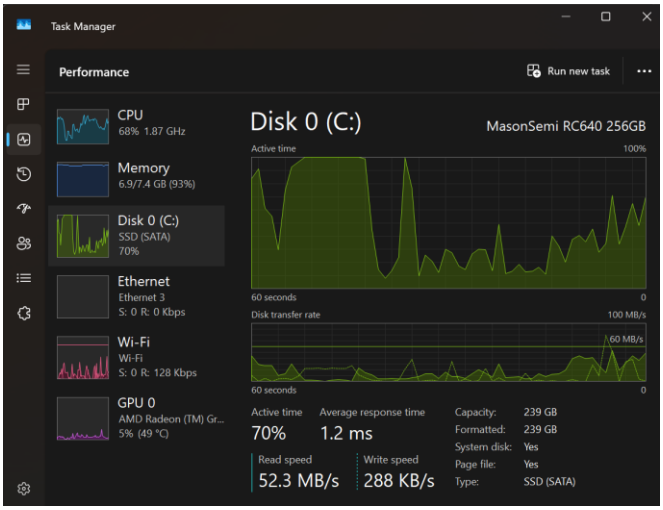
II. CPU usage at 53%, speed 1.85 GHz, 12 logical processors, Virtualization Enabled.



III. RAM usage 6.7 / 7.4 GB (91%), only 757 MB available.



IV. Disk active time 70%, read speed 52.3 MB/s, write speed 288 KB/s.



Resource Distribution:

When you run multiple virtual machines (VMs) on a single physical host, the hypervisor (VirtualBox) acts as a resource manager. It divides the host's physical hardware among the VMs according to their configured limits and current demand.

1. CPU Distribution

- Host has 12 logical CPU cores (AMD Ryzen 7430U).
- Each VM is assigned a certain number of virtual CPUs (vCPUs).
- VirtualBox uses the host CPU's hardware virtualization features (AMD-V) to allow VMs to execute instructions directly on physical cores.
- The hypervisor's scheduler gives each VM a time slice (e.g., 10–30 ms) of CPU time, then switches to another VM.

Result: Total host CPU usage was 53% – meaning the two VMs together consumed about half of the CPU's capacity. The remaining 47% was idle or used by the host OS.

2. RAM Distribution

- Host has 7.4 GB total RAM (after hardware reserved).
- Both VMs together consumed ~6.7 GB (91% of host RAM).
- This leaves only 757 MB for the host OS (Windows).

Why so high? Each VM reserves its configured RAM (e.g., 4 GB each). Even if the VM isn't actively using all of it, the hypervisor keeps that RAM allocated to ensure performance.

Risk: Low available memory can cause host swapping (using disk as slow memory), which explains the high disk usage (70%).

3. Disk Distribution

- Host disk (SSD) showed 70% active time and read/write activity.
- When host RAM becomes scarce, Windows starts using the page file (virtual memory) on disk.
- Also, VMs constantly read/write to their virtual disk files (.vdi).
- High disk usage indicates that the system is struggling with memory pressure - the host is forced to swap memory to disk, and VMs are competing for disk I/O.

LAB Task 12

Peripheral and Device Virtualization

Objective

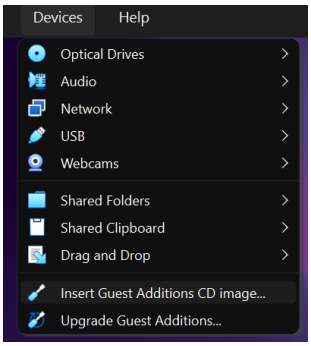
Enable host-guest communication using VirtualBox features:

- Shared Clipboard (Bidirectional)
- Drag and Drop (Optional)
- Shared Folder (persistent)

Step-by-Step Procedure

Install VirtualBox Guest Additions (if not already done)

Inside the running VM → Devices menu → Insert Guest Additions CD image → Run installer.



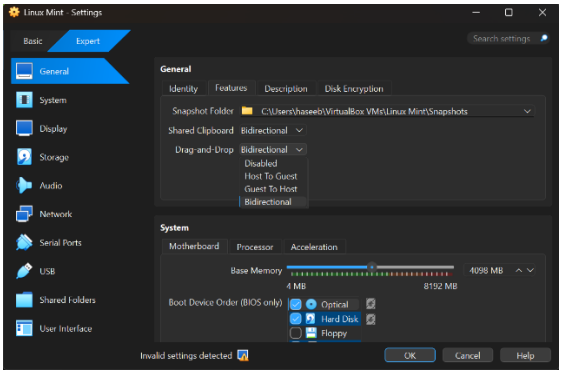
- Reboot the VM.

Enable Shared Clipboard & Drag and Drop

- With VM powered off (or running, but some settings require shutdown):

VirtualBox main window → Select your VM → Settings → General → Advanced tab.

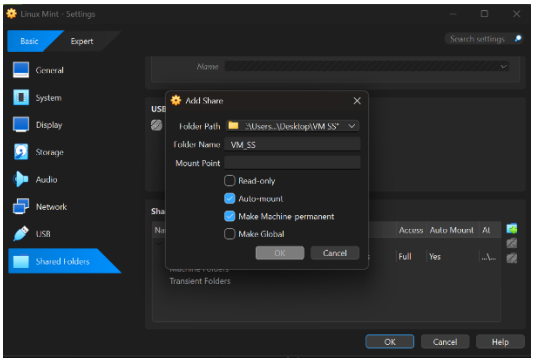
- Set Shared Clipboard to Bidirectional.
- Set Drag and Drop to Bidirectional (optional but recommended).
- Click OK.



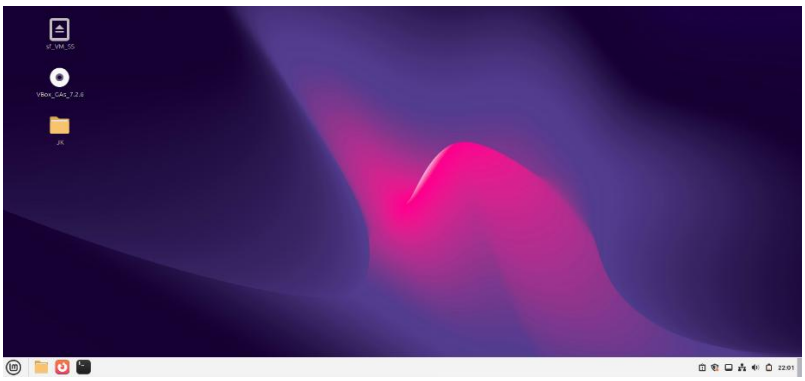
Add a Shared Folder

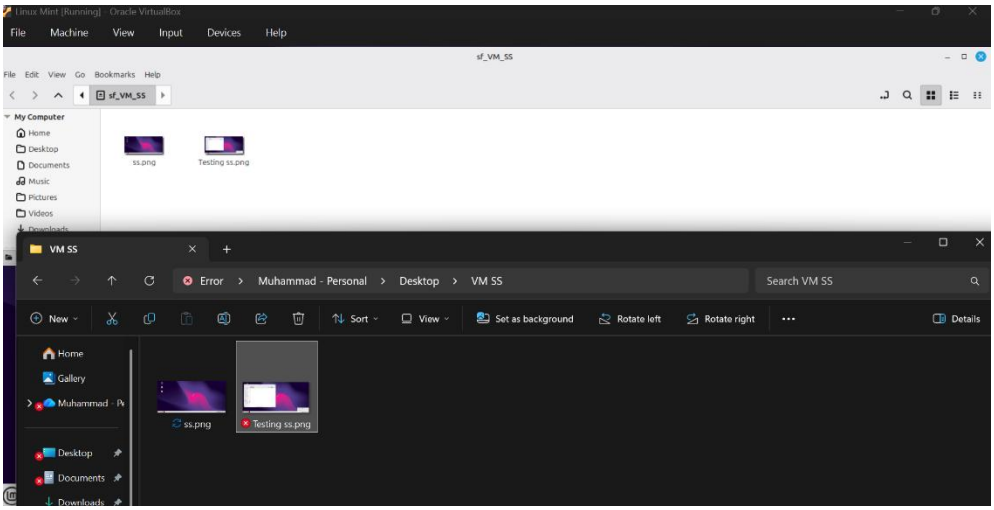
In VM Settings → Shared Folders → Click the + (add new shared folder).

- **Folder Path:** Choose a folder on your host (e.g., "C:\Users\haseeb\OneDrive\Desktop\VM SS"
- **Folder Name:** e.g., Vm_SS
- Check Auto-mount and Make Permanent (if available).
- Click OK.



Start the VM.





1. Shared Folder Purpose

A shared folder allows the host and guest operating systems to read/write the same set of files without using external drives, network transfers, or cloud storage. It behaves like a network drive but works through VirtualBox's Guest Additions feature.

2. How It Works

- VirtualBox Guest Additions installs a special file system driver inside the guest OS (Linux Mint).
- The hypervisor (VirtualBox) creates a virtual device that redirects file access requests between the host's real file system and the guest's virtual file system.
- When the guest writes to /media/sf_VM_SS, the hypervisor translates that write operation into a host file write to C:\VM_SS (or whatever host path you set).

3. Why sf_ prefix?

VirtualBox automatically mounts shared folders under /media/ with the name sf_<foldername>. The user (you) may need to be added to the vboxsf group to access it.

4. Verification in Your Screenshot

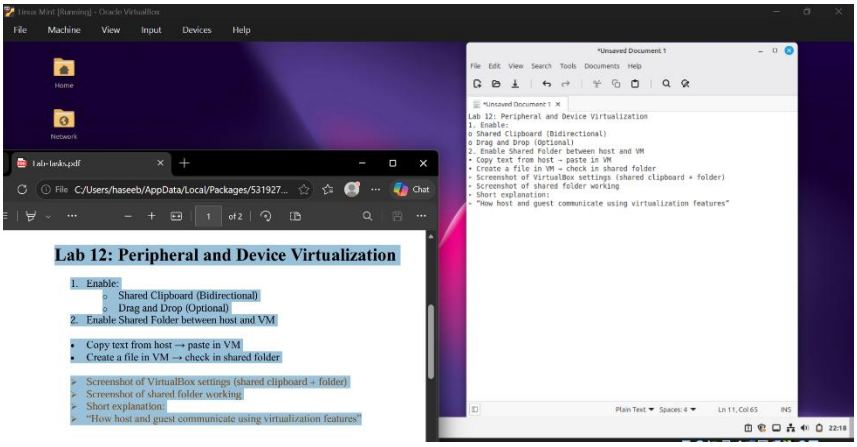
The file manager clearly shows sf_VM_SS as a location.

Two PNG files are visible. These could have been:

- Copied from the host into the shared folder (and now seen inside the guest), or
- Created inside the guest (e.g., using a screenshot tool) and automatically saved to the shared folder, making them accessible on the host.
- Either way, bidirectional file sharing is working.

Test Shared Clipboard

- Inside the VM, open a text editor.
- On the host, copy some text (e.g., from a browser).
- Paste into the VM's text editor.
- (Optional) Copy from VM and paste into host.



How Host and Guest Communicate Using Virtualization Features

Virtualization features (like those in Oracle VirtualBox) enable seamless communication between the host (your physical computer) and the guest (virtual machine). This is achieved through a combination of:

1. VirtualBox Guest Additions

Guest Additions is a set of device drivers and system utilities installed inside the guest OS. It optimizes the guest's performance and enables smooth interaction with the host.

2. Shared Clipboard (Bidirectional)

How it works: Guest Additions intercepts clipboard operations (copy/cut/paste) from both the host and guest. When you copy text on the host, the hypervisor passes that data to the guest's clipboard manager, and vice versa.

Result: You can copy text, URLs, or small images from the host and paste them directly inside the VM, and the other way around.

3. Drag and Drop (Bidirectional)

How it works: The hypervisor monitors mouse drag events across the VM window. When a file is dragged from the host into the VM window, VirtualBox captures the file's path, reads the file data, and recreates it inside the guest's file system using Guest Additions.

Result: You can drag files from your host desktop into the VM, or from the VM to the host, without external drives.

4. Shared Folders

How it works: The hypervisor "exposes" a directory from the host's file system to the guest as a virtual network drive or a mountable folder. The guest OS sees it as a regular folder (e.g., /media/sf_sharename). Read/write requests from the guest are intercepted by Guest Additions, translated, and sent to the host's file system driver.

Result: Both host and guest can read and write the same files in real time, as shown in your screenshot (sf_VM_SS).

5. Underlying Mechanism - Hypervisor as a Translator

The hypervisor acts as a middleman:

For shared clipboard & drag & drop: It transfers data buffers between the host and guest via a dedicated communication channel (virtual I/O port or shared memory).

For shared folders: It translates guest file system calls (e.g., open, read, write) into host operating system calls, using the Guest Additions driver as a bridge.

Note:

"Host-guest communication is enabled by VirtualBox Guest Additions, which intercepts clipboard, drag-and-drop, and file operations in the guest OS and translates them into host-side actions, allowing seamless data exchange without network or physical media."

LAB Task 13

VM Cloning and Export/Import

Objective

- Create a full clone of an existing VM.
- Export a VM to a portable OVA file.
- Import the OVA file back as a new VM.
- Understand the difference between cloning and export/import.

Step-by-Step Procedure:

Step 1: Take any existing VM

For the purposes of this lab, ensure you have a VM (for example, the Linux Mint VM from your previous tasks) that is fully shut down in VirtualBox's main window. It must not be in a "Saved" state.

Step 2: Create a Full Clone

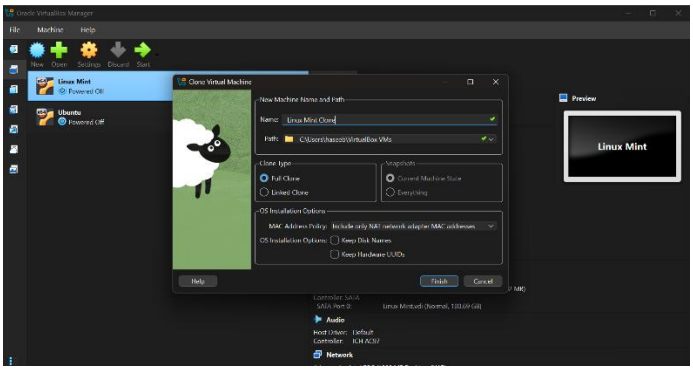
Cloning creates an independent, ready-to-run copy of a VM on the same host computer. A "Full Clone" creates a complete, separate copy of the VM's virtual disk, allowing it to operate independently of the source VM. It is ideal for creating a permanent, standalone duplicate, for example, for testing or development.

➢ Follow these steps to take your first screenshot ("Screenshot of cloning process"):

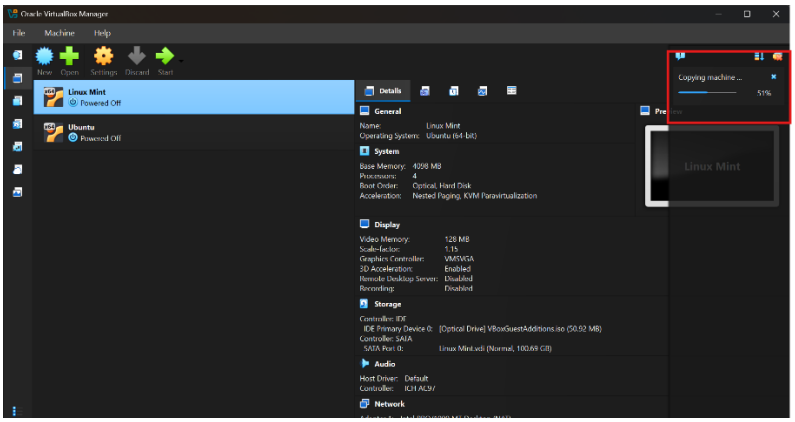
- In the VirtualBox Manager, right-click on your source VM and select "Clone".



- In the "Clone Virtual Machine" dialog box that opens, as shown in the example below, you will see several options:
- **Name:** Enter a name for your new VM (e.g., "Mint-Clone").
- **Path:** You can keep the default or choose a location with enough free space.
- **MAC Address Policy:** It's generally recommended to select "Generate new MAC addresses for all network adapters" to prevent network conflicts when both VMs are running.
- **Clone Type:** Select "Full clone".



Cloning VM:



- A new window titled "Clone Virtual Machine" in the Oracle VM VirtualBox Manager, showing the configuration options for creating a full clone of a virtual machine.
- Then, click "Clone" to start the process.

Step 3: Export VM as an OVA File

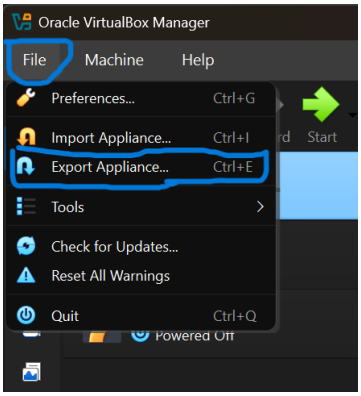
Exporting packages the entire VM (its configuration and disk images) into a single portable file, either an Open Virtualization Format Archive (OVA) file or a set of OVF files. This is the standard method for migrating a VM to a different virtualization platform or archiving it for later use.

What is an OVA?

An Open Virtualization Format Archive (OVA) is a single file (a .ova archive) containing all the data needed to recreate a virtual machine on another system, making migration simple.

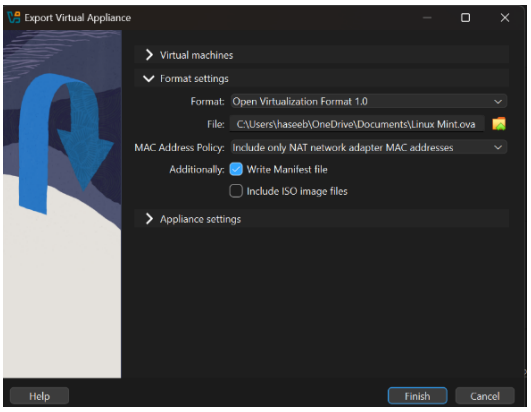
➢ Follow these steps to take your second screenshot ("Screenshot of exported file (.ova)"):

- Ensure your source VM is powered off.
- In the VirtualBox Manager menu bar, go to File → Export Appliance....
- Select the VM you want to export from the list and click Next.

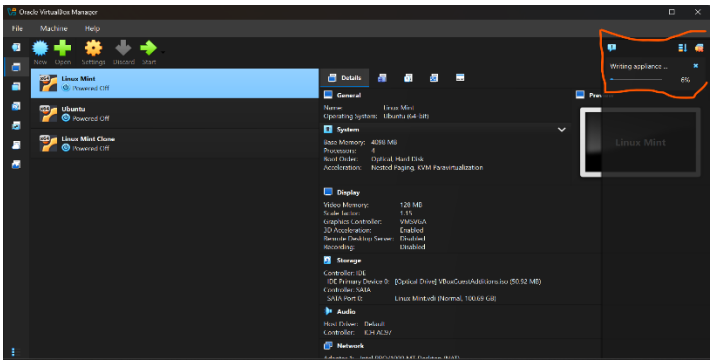


In the "Export Appliance Settings" window:

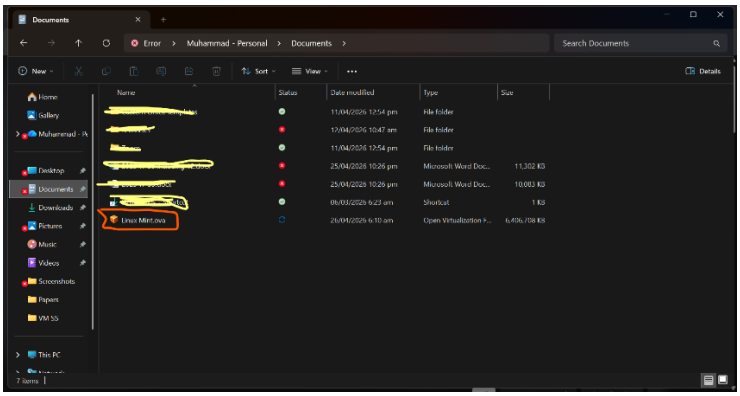
- **Format:** Choose "Open Virtualization Format Archive (OVA)" to create a single file.
- **File:** Select where you want to save the OVA file and give it a name (e.g., "MyVM.ova").
- Click Next.
- You can then review the VM settings. Leaving the defaults is generally fine.
- Click Finish to begin the export.



Exporting:



Once the export is complete, navigate to the location you saved the file. Check the .ova file in your file manager.

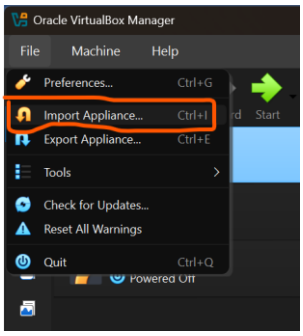


Step 4: Import the same OVA file back

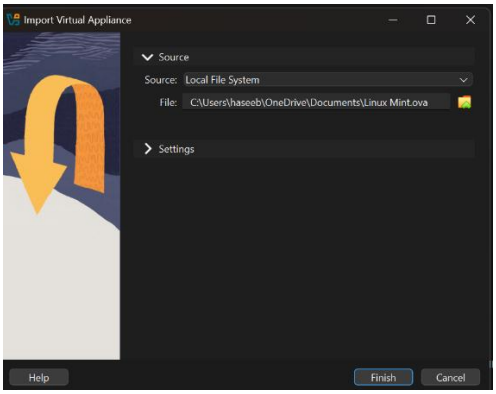
Importing is the process of taking an exported appliance file (an OVA file) and re-creating the virtual machine from it. This is how you can deploy a pre-configured VM on a new host.

➢ Follow these steps to take your third screenshot ("Screenshot of imported VM"):

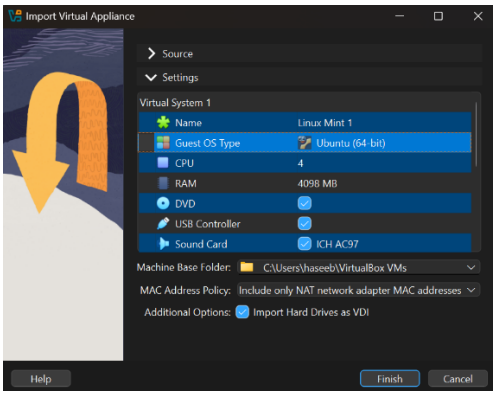
- In the VirtualBox Manager menu bar, go to File → Import Appliance....



- Click the folder icon and browse to select the .ova file you just exported.
- Click Next.



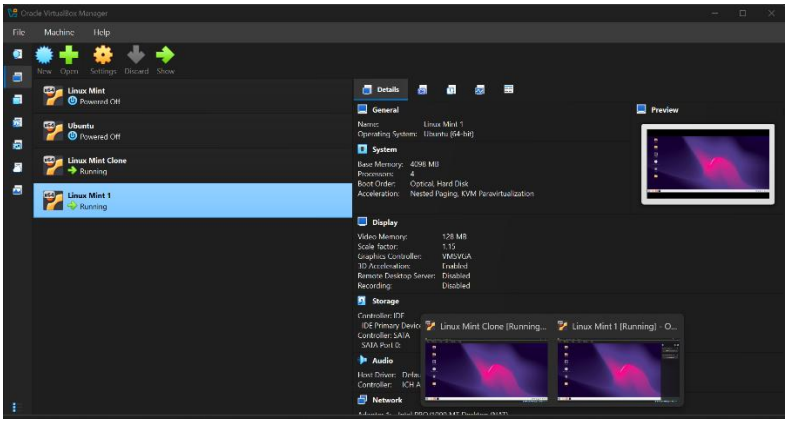
- In the "Settings" window, you can rename the VM to avoid confusion (for example, rename it to "Mint-Imported"). You can also change other settings like RAM, CPUs, or storage paths.



- Click Finish to start the import.

Once imported, you will see the new VM listed in the VirtualBox Manager.

Start both the clone (from Step 2) and the imported VM (using their new names) to check that they are working.



Explanation

Clone vs. Export/Import:

The main differences between these two operations are their purpose, output format, and where they are used:

Cloning a VM

- **Location:** Works on the same host computer.
- **Output:** It creates another native VirtualBox VM (a folder with .vbox and .vdi files) that is ready to run immediately.
- **Speed:** Generally very fast as it doesn't change the disk file format.
- **Use Case:** To quickly create another independent VirtualBox VM on the same machine for testing, development, or creating a backup copy.

Exporting and Importing a VM

- **Location:** Used to move a VM between different computers or to archive it.
- **Output:** It packages the VM into a standard, portable OVA (Open Virtualization Format Archive) file, which can be stored on a USB drive or uploaded to the cloud.
- **Disk Format Change:** During export, VirtualBox may convert the virtual disk to VMDK, the standard format used by VMware and other virtualization platforms.
- **Detail:** Because it re-packages the VM for portability, some specific hardware features may be reset or lost during the export/import process.

- **Use Case:** To distribute a VM to colleagues, migrate it to a data center, or create a long-term, hardware-independent backup.

LAB Task 14

Suspend, Resume and VM State Management

Objective

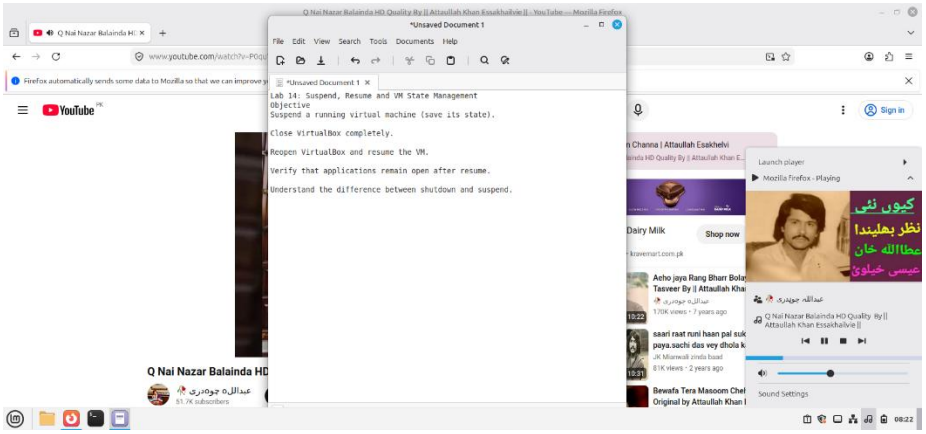
- Suspend a running virtual machine (save its state).
- Close VirtualBox completely.
- Reopen VirtualBox and resume the VM.
- Verify that applications remain open after resume.
- Understand the difference between shutdown and suspend.

Step-by-Step Procedure

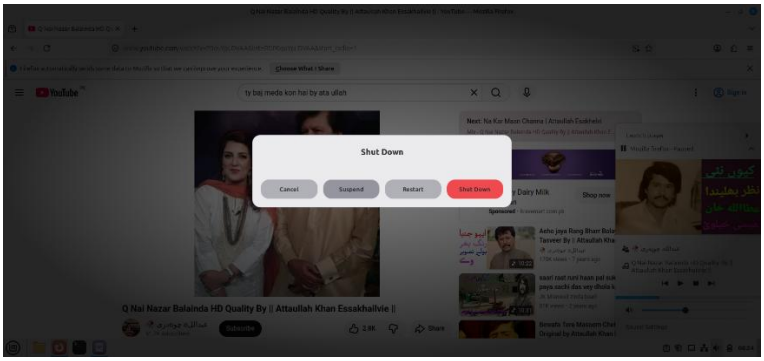
Start your VM (e.g., Linux Mint or Ubuntu).

Open a few applications inside the VM:

- Text editor (e.g., gedit or xed) - type something unsaved to test.
- Terminal (leave it open).
- Web browser (navigate to any page).

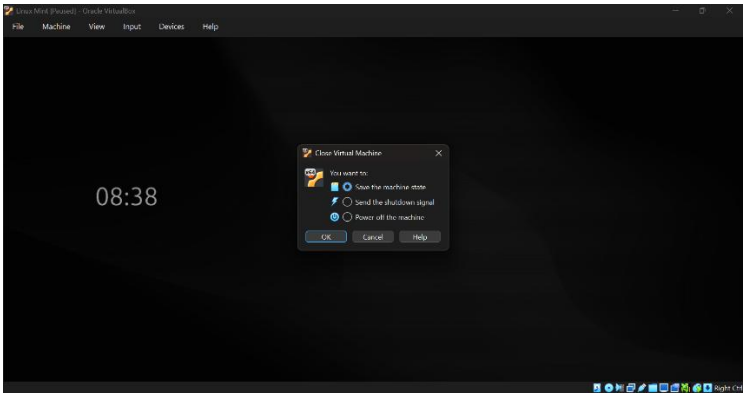


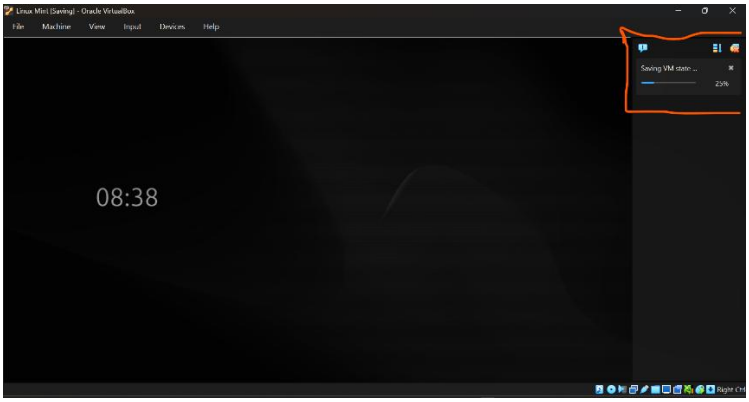
Suspend the VM (save its state):

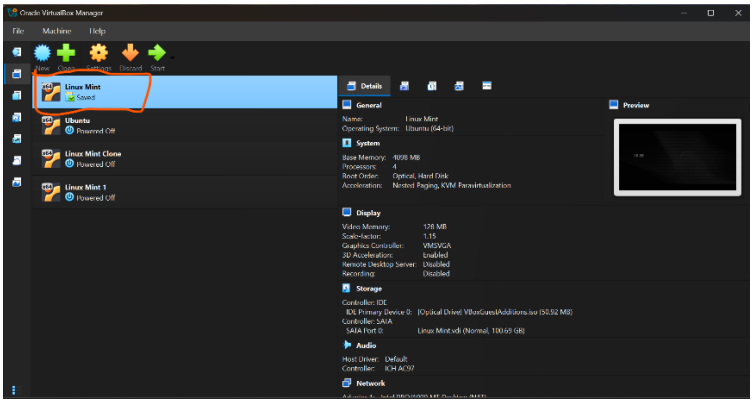


From the VM menu: Machine → Close → Save the machine state.

- Or click the X (close button) of the VM window and select Save the machine state.

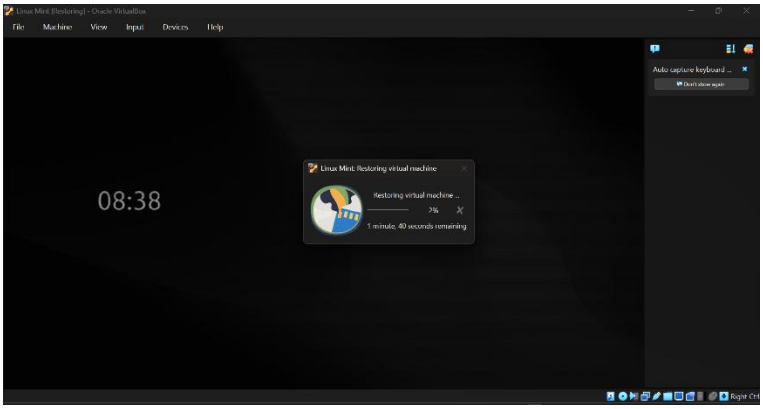




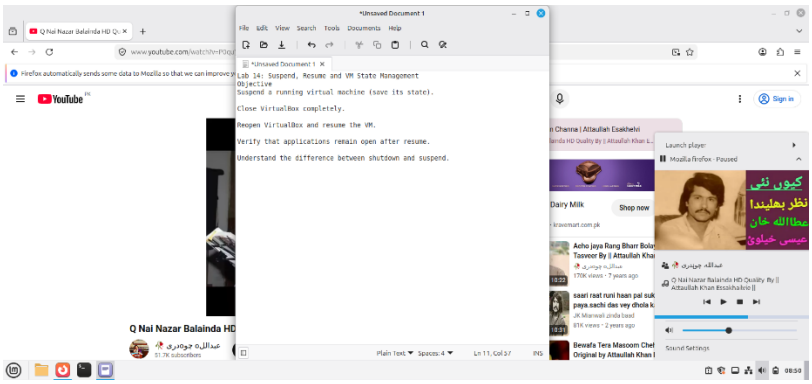


- Close VirtualBox completely (exit the VirtualBox manager).

Reopen VirtualBox and click on saved VM(Linux Mint).



- Start the same VM - it should resume exactly where you left off, with all apps still open and unsaved text still present.
- Check - verify that the text editor still has your unsaved text, the terminal shows the same command history, and the browser is on the same page.



What is VM state saving and why it is useful?

VM state saving (suspend) captures the entire current state of the virtual machine - including RAM contents, CPU registers, and open applications - and stores it on the host disk. When resumed, the VM continues exactly where it left off without rebooting. This is useful for quickly pausing work without losing progress, saving power, or freeing host resources temporarily while preserving the exact working environment.

Comparison: Shutdown vs Suspend

Shutdown:

When you shut down a VM, the guest operating system goes through a complete power-off sequence. All running processes - including your open applications like the terminal, text editor, or browser - are terminated gracefully (or forcibly) by the OS. The virtual machine's RAM is

cleared, and no execution state is saved. As a result, the next time you start the VM, you must go through the full boot process: BIOS/UEFI initialization, bootloader, kernel loading, login screen, and then manually reopen your applications. Any unsaved work inside those applications is lost unless you saved it to disk beforehand. Shutdown is useful for cleaning up the VM, saving resources permanently, or preparing the VM for cloning or export.

Suspend:

When you suspend a VM (also called "Save the machine state"), the hypervisor (VirtualBox) captures the exact contents of the VM's RAM, CPU registers, and device states, then writes all that data into a state file (.sav) on the host computer's disk. The VM then stops running. Because the state is frozen and stored, when you later resume the VM, the hypervisor reads that file back into memory and restores everything exactly as it was. There is no boot process - the VM simply continues from the exact moment you suspended it. All your applications remain open with the same data, unsaved text in editors stays intact, and you can resume work within seconds. The trade-off is that the state file uses disk space equal to the VM's assigned RAM (e.g., 4 GB RAM = 4 GB state file), and if you move the VM to another machine without the state file, the saved state cannot be restored.

Summary:

Shutdown gives you a clean, fresh start at the cost of time and lost working context, while suspend preserves everything for instant resumption at the cost of disk space and dependency on the same host environment.

LAB Task 15

Comparison Table of Local VM and Cloud VM

Objective:

Understand the key differences between a Local Virtual Machine (running on your own computer using VirtualBox/VMware) and a Cloud Virtual Machine (running on a provider's infrastructure like AWS EC2, Azure, or Google Cloud).

Comparison Table:

Feature	Local VM (Your PC)	Cloud VM (AWS/Azure/GCP)
Upfront Cost	**Free** (You already own the hardware).	**Variable** (Pay-as-you-go per hour/minute).
Performance	Limited by your i5 CPU and 16GB RAM.	**Scalable** (Can rent 128GB RAM or 64 cores).
Internet Need	None (Works fully offline).	**Required** (You must be online to access it).
Storage	Limited by your **256GB SSD**.	Virtually unlimited (but you pay per GB).
Setup Speed	Minutes (Installing an ISO).	Seconds (Deploying a pre-made image).
Reliability	Depends on your laptop's power/battery.	High (99.9% uptime in data centers).
Privacy	High (Data stays on your physical SSD).	Managed by provider (Data is on their servers).

Summary:

Local VMs are ideal for learning, offline development, and testing because they cost nothing extra, work without internet, and keep data private. Cloud VMs offer scalability, high availability, and fast deployment - but require internet, incur ongoing costs, and involve trusting the provider with your data. The choice depends on your needs: local for control and privacy, cloud for scale and reliability.

LAB Task 16

Linux Commands - Local VM vs Cloud VM

Objective:

Learn 5 common Linux commands that work in both Local and Cloud VMs, and understand how their usage differs between the two environments.

The 5 Commands

#	Command	Basic function
1	ssh	Securely connect to a remote machine
2	scp	Copy files between machines over SSH
3	systemctl	Control system services (start, stop, enable)
4	df -h	Check disk space usage
5	htop / top	Monitor real-time CPU and memory usage

Explanation with Usage Differences (Local VM vs Cloud VM)

1. SSH - Secure Shell

SSH is a protocol used to securely access a remote machine through a command-line interface.

Local VM:

- Usually not required because you already have direct console access via tools like VirtualBox or VMware.
- You can open the terminal directly inside the VM.
- SSH is only used if you want remote-style access.

Example:

- ssh localhost

Cloud VM:

- Essential command because there is no physical access to the machine.
- Used to connect from your local computer to the cloud server over the internet.

Example:

- ssh -i key.pem ubuntu@54.123.45.67

Key Difference:

- Local VM → Optional (direct access available)
- Cloud VM → Mandatory (only remote access available)

2. SCP - Secure Copy

SCP is used to transfer files securely between machines using SSH.

Local VM:

Can be used if networking is configured.

However, easier methods like shared folders or drag-and-drop are commonly used.

Example:

- scp file.txt user@192.168.1.100:/home/user/

Cloud VM:

- One of the main file transfer methods.
- Used to upload code, configuration files, or download logs.

Example:

- scp -i key.pem ubuntu@54.123.45.67:/var/log/nginx/access.log .

Key Difference:

- Local VM → Optional (other easier methods exist)
- Cloud VM → Primary file transfer method

3. systemctl - Service Management

systemctl is used to manage system services (start, stop, restart).

Local VM:

Used normally to control services like web servers or databases.

Example:

- sudo systemctl start apache2

Cloud VM:

Works exactly the same.

However, cloud systems may include extra background services (agents).

Example:

- sudo systemctl stop cloud-init

Key Difference:

- Command is identical
- Cloud VM includes additional cloud-specific services

4. df -h - Disk Space Check

This command shows disk usage in a human-readable format.

Local VM:

Displays usage of the virtual disk assigned to the VM.

Example:

- df -h

Cloud VM:

Shows multiple storage types:

- Persistent storage (e.g., main disk)
- Temporary (ephemeral) storage
- Additional attached volumes

Example:

- df -h

Key Difference:

- Local VM → Single virtual disk
- Cloud VM → Multiple storage types (persistent + temporary)

5. htop / top – Resource Monitoring

These tools show CPU, memory, and running processes.

Local VM:

- Displays resource usage based on allocated hardware from host system.
- What you see is what the VM is actually using.

Example:

- htop

Cloud VM:

Shows only inside-VM (guest OS) usage.

Does NOT show:

- CPU throttling
- Shared resource contention

- For full monitoring, cloud dashboards are also used.

Example:

- htop

Key Difference:

- Local VM → Direct and accurate view of allocated resources
- Cloud VM → Limited view (external monitoring also required)

Summary Table (For Lab Manual)

Command	Local VM Usage	Cloud VM Usage	Key Difference
ssh	Optional	Mandatory	Access method
scp	Alternative	Primary	File transfer reliance
systemctl	Same	Same + extra service	Cloud agents
df -h	Virtual disk	Multiple volumes	Storage types
htop	Full VM view	Guest-only view	No host insight

Summary:

These Linux commands behave the same at the operating system level in both local and cloud virtual machines. However, their usage differs in context: cloud VMs depend on SSH for access, SCP for file transfer, and include additional services, storage types, and external monitoring tools that are not typically present in local VMs.